

# Security Below the OS – A Security Analysis of UEFI

PRIYANKA PRAKASH SURVE, Ben-Gurion University of the Negev, Israel

OLEG BRODT, Ben-Gurion University of the Negev, Israel

MARK YAMPOLSKIY, Auburn University, United States

YUVAL ELOVICI, Ben-Gurion University of the Negev, Israel

ASAF SHABTAI, Ben-Gurion University of the Negev, Israel

The Unified Extensible Firmware Interface (UEFI) plays a crucial role in modern computing systems, governing secure system initialization and booting. A sharp spike in UEFI-related attacks and vulnerabilities has been observed in recent years [21]. To respond to this alarming trend, we believe that the cybersecurity community should be equipped with the knowledge to understand the UEFI landscape, the related attacks, and potential defenses. Over the years UEFI has been a niche topic in cybersecurity, attracting little research and lacking a comprehensive overview. The knowledge that exists on the topic is scattered across official documentation, blog posts, papers, and books which are not necessarily security focused, creating a barrier to entry for cybersecurity professionals who want to learn about the topic. This paper aims to correct that problem by exploring the UEFI from a security point of view, including the UEFI landscape, the UEFI development lifecycle, distribution, supply chain, and booting process. We investigate a wide range of real-world UEFI attacks and potential attack vectors that could exploit vulnerabilities at the various stages of the UEFI lifecycle that we examine in this paper. Inspired by the MITRE ATT&CK framework, we present a taxonomy delineating the tactics, techniques, and sub-techniques of UEFI attacks based on the commonalities and patterns observed in our examination of the attacks. Further to outlining attacks and defenses, we perform a risk analysis to identify the techniques that pose the greatest threat to UEFI security. Prioritizing the most critical risks will enable future research aimed at addressing the most vulnerable aspects of the UEFI and contributing to its security.

CCS Concepts: • **Security and privacy** → **Operating systems security**; • **Computer systems organization**;

Additional Key Words and Phrases: UEFI, UEFI Lifecycle, UEFI Attacks, countermeasures, Risk Analysis

## 1 Introduction

UEFI is located on the SPI chip of the motherboard and it is the first code that runs after the computer is switched on. UEFI specifications define the interface between the computer hardware and the operating system (OS) on modern computer systems and provide a standardized way for initializing the computer hardware, booting the operating system, and providing runtime services [107]. Throughout this paper, the term *UEFI* refers specifically to the Unified Extensible Firmware Interface and its associated firmware. The term *BIOS* is used only when referring to legacy Basic Input/Output System firmware predating UEFI, or when citing vendor or tool terminology that uses BIOS to refer to the firmware flash chip generically, as is common in industry practice.

UEFI offers enhanced security features as compared to traditional Basic Input/Output Systems (BIOS), but despite continuous enhancements and advancements in UEFI security it is increasingly becoming a lucrative target for cyber threat actors [58]. This trend is largely due to the complexity of (i) the UEFI supply chain, where multiple actors, including hardware manufacturers, software developers, and end-users play an important role at different stages of the UEFI development and maintenance life cycle (yet their responsibilities often overlap and extend beyond a single phase) [8]; and (ii) the architecture of UEFI itself, which is designed to provide a more flexible and feature-rich environment for booting systems than the legacy BIOS, through a phased booting process where

---

Authors' addresses: Priyanka Prakash Surve, [surve@post.bgu.ac.il](mailto:surve@post.bgu.ac.il), Ben-Gurion University of the Negev, Be'er Sheva, Israel; Oleg Brodt, [boleg@bgu.ac.il](mailto:boleg@bgu.ac.il), Ben-Gurion University of the Negev, Be'er Sheva, Israel; Mark Yampolskiy, [mark.yampolskiy@auburn.edu](mailto:mark.yampolskiy@auburn.edu), Auburn University, United States; Yuval Elovici, [elovici@bgu.ac.il](mailto:elovici@bgu.ac.il), Ben-Gurion University of the Negev, Be'er Sheva, Israel; Asaf Shabtai, [shabtaia@bgu.ac.il](mailto:shabtaia@bgu.ac.il), Ben-Gurion University of the Negev, Be'er Sheva, Israel.

control is handed off from one phase to another, and through the support of multiple drivers and functionalities. The complexity of the boot process and the integration of various drivers and functionalities increase the attack surface, providing adversaries with numerous opportunities to exploit vulnerabilities [24].

Recent years have witnessed a sharp increase in UEFI-related attacks reflected in the growing number of UEFI-specific entries in CISA's Known Exploited Vulnerabilities catalog [21], with notable examples discussed in the following chapters. The challenge of detecting and removing threats at the UEFI level highlights the need for specialized security solutions and practices designed to protect against, and respond to, such deep-seated attacks.

### 1.1 Scope and Outline

We begin by exploring UEFI's functionality and inherent security features in Section 2 and then study the UEFI-related attacks found in the wild such as BlackLotus, MoonBounce, FinSpy, etc. in Section 3. Following this study of past attacks, in Section 4 we present a MITRE-like [75] taxonomy that includes the tactics, techniques, and sub-techniques employed to carry out UEFI-related attacks. In Section 5 we create a taxonomy of countermeasures that can be used to protect the system against UEFI-related attacks. We conclude the paper by performing a risk analysis of the UEFI security landscape in Section 6 to find out the most vulnerable components of the UEFI ecosystem.

### 1.2 Related Work

The early investigation into the vulnerabilities within UEFI firmware by Bashun et al. in "Too Young to Be Secure: Analysis of UEFI Threats and Vulnerabilities" (2013) [6] set the stage for subsequent research in this relatively uncharted field. Building on the foundational concerns, Rodionov et al.'s "Bootkits: Past, Present & Future" (2014) [89] traced the evolution of rootkits, including those targeting UEFI. This survey provided an understanding of how UEFI threats have progressed over time, and indicated a trend toward more sophisticated and targeted attacks. Diving deeper into specific threats, Wang and Dong's "Attacking Intel UEFI by Using Cache Poisoning" (2019) [109] gave details about a novel attack method, illustrating the advanced nature of threats to UEFI systems. This specificity in research pinpointed vulnerabilities that could be exploited by attackers with a high degree of technical knowledge. With the threat landscape evolving, recent studies have focused on proactive security enhancements. Krichanov and Cheptsov's work on "UEFI Virtual Machine Firmware Hardening Through Snapshots and Attack Surface Reduction" (2021) [67] introduced a methodological approach to defend against UEFI threats in virtualized environments. This work demonstrated the potential of leveraging virtualization technology to safeguard UEFI firmware. In a more recent setting, Jiao et al.'s "UEFI Security Threats Introduced by S3 and Mitigation Measures" (2022) [57] targeted the vulnerabilities present in the S3 sleep state, offering mitigation strategies tailored to the nuances of power state transitions in UEFI. This indicates a growing awareness of the need to secure all aspects of UEFI functionality. The most recent survey by Zhou et al., "A Survey on the Evolution of Bootkits Attack and Defense Techniques" (2024) [116], along with a practical exploration of rootkit design in "The Design of the Simple SMM Rootkit" (2022) [98], collectively indicate a sustained interest in UEFI security. These works contribute to a broader understanding of how attackers may exploit UEFI, and the defensive techniques that could be employed in response.

Despite the progress made in identifying and mitigating UEFI security threats, the field remains under-researched. The existing literature, while providing isolated insights, falls short of establishing a comprehensive taxonomy of UEFI-related attacks. Such a taxonomy is essential for several reasons—it serves as a foundation for recognizing common patterns in attacks which could be an essential building block for organizations' Security Information and Event Management (SIEM) and Threat Hunting systems; facilitates the systematic arrangement of countermeasures; and

enables the identification of the most exploitable attack vectors. Hence, there is a pressing need for a robust taxonomy of UEFI-related attacks. The lack of such a taxonomy hinders the ability to conduct a thorough risk analysis, which is critical for determining the most exploitable weaknesses within UEFI systems. The countermeasures against UEFI threats are often discussed in isolation, without the context of an overarching defense strategy. A structured presentation of these defenses, correlated with a detailed taxonomy of attack techniques, is essential for a holistic understanding of UEFI security and the efficacy of various protection mechanisms. This research addresses a gap in the existing research on UEFI security by presenting a comprehensive framework. The framework encompasses a taxonomy of UEFI threats, a systematic arrangement of countermeasures, and a risk analysis that identifies and prioritizes the most critical vulnerabilities. The objective is to guide future research and development endeavors, focusing on fortifying the most vulnerable aspects of UEFI security.

### 1.3 Contributions

The main contributions of this paper can be summarized as follows:

- (1) *Formalization of UEFI Security Knowledge*: Systematically formalizing existing UEFI security knowledge by examining its history, development lifecycle, supply chain, and functionalities.
- (2) *Taxonomy of UEFI-Related Attacks*: Developing a UEFI-specific taxonomy of attack tactics and techniques, mapping the attack landscape based on existing literature, and presenting it in a MITRE-like layout for ease of use.
- (3) *Taxonomy of UEFI-Related Security Controls*: Enumerating and categorizing security controls designed to defend against UEFI-related attacks, and providing a reference guide for potential threats.
- (4) *Comprehensive Risk Analysis*: Performing a thorough risk analysis that identifies defense gaps and suggests urgent research directions to enhance UEFI security.

## 2 Background

Since the inception of computer systems, the Basic Input/Output System (BIOS) has been essential for hardware initialization during the boot process [101]. As the first code that runs in a computer system, its primary tasks are to initialize the hardware and launch the OS. However, with advancements in computing, several limitations of legacy BIOS have become apparent. These include the inability to boot from hard drives larger than 2.2 TB, slow boot times, the absence of inherent security mechanisms, a fairly simplistic user interface, and a lack of extensibility due to its monolithic design and CPU-dependent architecture [9, 118].

To address these shortcomings, the UEFI Forum [108]—an industry-wide group comprising hardware manufacturers, major chip manufacturers, motherboard manufacturers, and OS vendors—released the Unified Extensible Firmware Interface (UEFI) specifications in 2005, aimed at fostering the development of a successor to legacy BIOS.

### 2.1 Components Involved in UEFI Boot

The following components are involved in the UEFI boot process [105]:

**2.1.1 SPI Flash Chip**: The Serial Peripheral Interface (SPI) flash chip, often referred to as the BIOS or firmware flash, is a non-volatile storage medium residing on the motherboard. It stores firmware that initializes hardware and provides services for the system's software [112]. The exact contents of the SPI chip vary depending on system manufacturers, motherboard or device models, UEFI firmware vendors, and system configuration:

- *UEFI Firmware*: This is the primary content of most SPI flash chips on PCs. It initializes hardware, provides low-level services, and then hands off control to the operating system. UEFI uses the physical flash device as storage. UEFI firmware consists of firmware volumes, which contain firmware files, and these files contain the most basic unit—the firmware segment.
- *UEFI Variables and Configuration Data*: On some systems, UEFI variables (such as boot order, security keys for Secure Boot, and other configuration settings) are stored in a dedicated region of the SPI flash chip. Not all UEFI variables are hard-coded onto the SPI chip. Many UEFI variables, such as boot order and system settings, are dynamically set and modified during system boot or other run-operations. Some variables, especially those related to system security, like Secure Boot keys, might be provisioned during manufacturing and are not intended to be changed under normal circumstances [55]. A few common such variables are:
  - (1) *Platform Key (PK)*: Used in the Secure Boot process to verify the Key Exchange Key (KEK) and control its updates.
  - (2) *Key Exchange Key*: Used to verify db and dbx updates.
  - (3) *Allowed Signatures Database (db)*: Contains keys and signatures of allowed bootloaders and drivers.
  - (4) *Forbidden Signatures Database (dbx)*: Contains revoked certificates of forbidden bootloaders and drivers in the form of their hashes.
- *System Management Mode (SMM) Code*: SMM is a special operating mode in x86 microprocessors introduced by Intel as a security and system-management feature. It provides an isolated environment for system-level operations that are transparent to the operating system, applications, and other components. It handles system-wide functions like power management, hardware control, or proprietary OEM-designed code. Code executed in this mode can be stored on the SPI flash chip [98].
- *Option ROM*: These firmware pieces help initialize specific hardware, such as graphics cards or certain storage controllers.
- *Recovery Firmware*: Some systems include a backup or recovery version to restore the firmware if the primary copy becomes corrupted [79].
- *Certificates and Keys*: For systems that support Secure Boot or other security features, necessary certificates and keys might be stored in the SPI flash [79].
- *Diagnostic Tools or Utilities*: Some manufacturers include built-in diagnostic tools or utilities in the firmware, accessible during boot [118]. Although these components are primarily intended for maintenance and troubleshooting, if they are exposed insecurely or insufficiently protected, they may expand the pre-OS attack surface and provide additional functionality that could be abused by an attacker.

**2.1.2 Serial Peripheral Interface (SPI)**: It is a synchronous serial communication interface used for short-distance communication, primarily in embedded systems [5]. The SPI flash chip communicates with the CPU using the SPI protocol over an SPI Bus. When the system powers on, the CPU communicates with the SPI flash chip through the SPI controller to fetch the initial firmware code for execution. This bootstrapping process begins when the CPU starts the system's firmware, which initializes the rest of the system and eventually loads the operating system [113].

**2.1.3 EFI System Partition (ESP)**: The ESP is a crucial component of the UEFI firmware architecture. Its primary purpose is to hold the bootloader and related files necessary for the computer to boot. The ESP is a dedicated partition on a storage device, typically a hard drive or solid-state device (SSD), that stores essential boot-related files and is created during the installation of the operating

system. The installer typically creates a small, dedicated partition at the beginning of the storage device. It formats this partition with the FAT32 file system and assigns the EFI System Partition label to it. Once the ESP is created, the installer copies the necessary bootloader and related files to it. The installer often configures the UEFI bootloader to point to the location of the OS files on the storage device. This configuration is stored in the boot configuration data (BCD) within the ESP [72]. On machines with multiple operating systems, each OS typically stores its bootloader in the ESP. The UEFI firmware then presents a boot menu, allowing the user to choose which OS to start.

If Secure Boot is enabled, the installer may also generate and install Secure Boot keys and certificates. These keys verify the authenticity of the bootloader and other crucial components during the boot process [48].

## 2.2 UEFI Boot Process

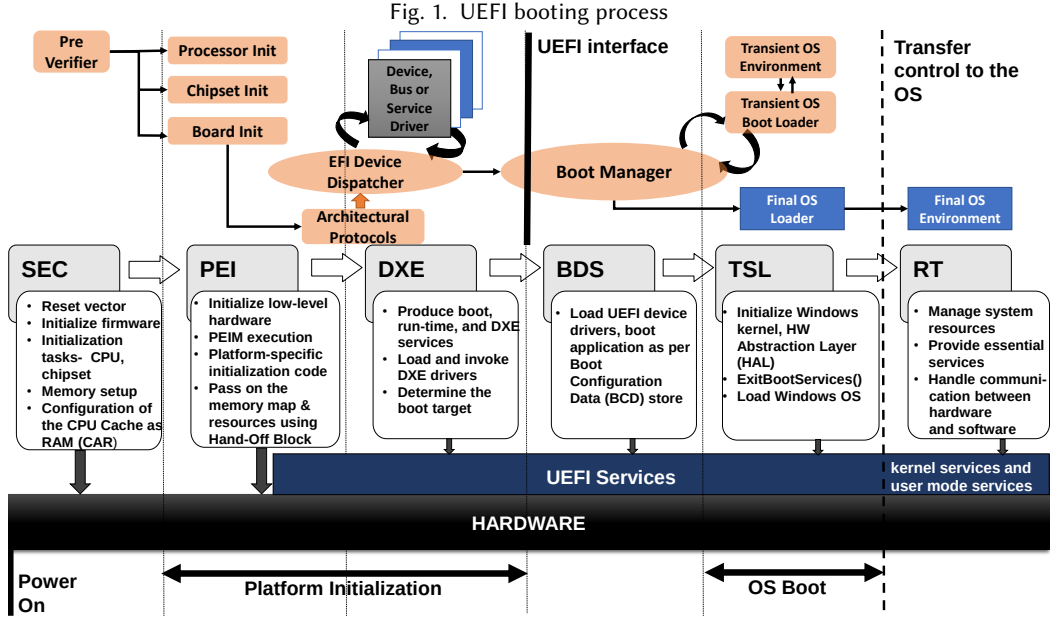
The UEFI standard specifies a flexible environment for the pre-boot and boot processes in computer systems. It outlines the essential features and requirements for UEFI firmware, covering crucial aspects like booting, hardware initialization, and run-time services [106]. The UEFI PI (platform initialization) is a subset of the UEFI specification that defines a standardized set of boot and run-time services for platform initialization [105]. While the services and their interfaces are standardized, UEFI implementations can differ across computer manufacturers. Each manufacturer typically develops its own UEFI firmware based on a reference implementation [3].

From a security perspective, the boot process consists of two distinct parts. The first is the platform initialization process, independent of the OS installed on the system [105]. It starts when the system powers on and continues until the execution of DXE phase is completed, after which control is handed from UEFI to the target OS [119]. The second part is specific to the OS being chosen for boot, differing for Windows, Mac, Linux, etc. [72, 106]

According to the UEFI specification, the entire system boot process consists of six phases, as depicted in Figure 1, during which different tasks are performed. Of these, the following phases are OS-independent:

- (1) **SEC Phase:** The SEC phase begins immediately after the system powers on or resets. It starts by running UEFI code from the SPI chip on the motherboard. During this phase, the CPU and chipset are initialized, and system memory is set up. The SEC phase also includes configuring of the CPU cache as RAM (CAR), establishing a temporary memory store. CAR then facilitates the execution of subsequent UEFI firmware initialization stages [105].
- (2) **Pre-EFI Initialization (PEI) Phase:** Various PEI modules (PEIMs) are executed during the PEI phase to handle specific initialization tasks. These modules include platform-specific initialization code, memory initialization routines, and CPU initialization. The last PEIM executed during this phase is DXE IPL (Initial Program Load), which handles the transition to the DXE phase [105].
- (3) **Driver Execution Environment (DXE) Phase:** Most of the system initialization takes place in the DXE phase. During this phase, device drivers are loaded. It also creates tables for EFI boot and run-time services.
- (4) **Boot Device Selection (BDS) Phase:** In this phase, the UEFI boot manager hands control to the OS boot manager selected by the user (e.g., *bootmgfw.efi* for Windows). After selecting the OS boot manager, boot behavior is determined by settings stored in the Boot Configuration Data (BCD) file. The boot manager then loads the boot loader, such as *winload.efi* for Windows. For execution, *winload.efi* also relies on the information in the BCD. All of these

files (*bootmgfw.efi*, *BCD*, *winload.efi*) reside in the EFI System Partition (ESP) of the hard drive.



- (5) **TSL Phase:** OS kernel initialization takes place in this phase. When initialization is complete, the *ExitBootServices()* function (from UEFI boot services) is called, which marks the completion of the boot process. After executing *ExitBootServices()*, only UEFI run-time services remain available, while UEFI boot-time services are no longer accessible [106].
- (6) **Run-Time:** After the kernel initialization is complete in the TSL phase, control is handed over to the OS run-time phase [72].

### 2.3 Security Features of UEFI

UEFI provides several security features to protect the integrity of the boot process and OS components. These features aim to enhance the security of the boot process, firmware, and overall system integrity. This section explores some key security features available in UEFI.

**2.3.1 Secure Boot:** The primary objective of Secure Boot is to ensure that only trusted software loads during the boot process, preventing the execution of unauthorized or malicious code [15]. Secure Boot verifies digital signatures to validate the integrity and authenticity of the boot components. This mechanism safeguards against malware that may attempt to exploit the boot process [72]. Microsoft bootloaders, being Secure Boot-aware, leverage UEFI Secure Boot keys and databases. During the boot process, these bootloaders verify the digital signatures of firmware, drivers, and OS components against keys stored in the UEFI firmware. This verification ensures that only trusted and signed components load, preventing the execution of unauthorized or malicious code.

Linux bootloaders follow a different approach to key management for Secure Boot. They rely on the machine owner key (MOK) and shim to switch to a key chain provided by Red Hat or Canonical, rather than using the UEFI variables directly. This allows Linux distributions to use their own set of keys and enable Secure Boot functionality without relying solely on UEFI firmware keys [54].

This approach allows Linux distributions to control the Secure Boot process and provide a Secure Boot environment for users.

Both approaches—Microsoft bootloaders leveraging UEFI Secure Boot keys and Linux bootloaders using MOK and Shim—aim to verify the integrity and authenticity of boot components.

**2.3.2 *Secure UEFI Variable Storage:*** UEFI incorporates a robust storage mechanism to secure variables. These variables are safeguarded against unauthorized modifications by external parties [84].

**2.3.3 *Platform Key and Signature Database:*** UEFI firmware provides native support for cryptographic keys, including PK and KEK. These keys play a vital role in verifying the integrity and authenticity of boot components [112]. The PK holds the highest significance, serving as a root of trust within the system. KEKs facilitate the management of other keys within the UEFI environment [55]. UEFI firmware also maintains a signature database that houses certificates and hashes of trusted boot components [112]. These databases serve as references for validating the authenticity and integrity of boot loaders, drivers, and firmware updates.

**2.3.4 *Credential Storage:*** UEFI firmware can securely store credentials, such as Secure Boot keys or BitLocker recovery keys, to safeguard against unauthorized access. By securely storing these credentials, UEFI firmware ensures that only authorized entities can retrieve and use them, preventing unauthorized access to critical system components or data [85].

**2.3.5 *Trusted Platform Module (TPM) Integration:*** The TPM is a dedicated cryptographic hardware module that enhances system security. When integrated with UEFI, TPM offers security capabilities, including secure key storage, Secure Boot measurements, and remote attestation [100].

**2.3.6 *Measured Boot:*** Measured Boot is a UEFI feature that captures a cryptographic hash, or measurement, of every component loaded during the boot process [79]. These measurements can be securely stored in the TPM and used for attestation purposes, thereby ensuring the security of the boot process. During each boot, Measured Boot stores the cryptographic hash of each loaded component, including boot loaders, firmware, and OS files. These measurements are securely stored within the TPM, providing a tamper-resistant environment for their retention. The stored measurements can be accessed later for attestation, which enables a trusted entity to remotely verify the integrity and security of the boot process. By comparing stored measurements with expected values, the system can provide evidence of a Secure Boot process, guarding against tampering or unauthorized modifications [78].

**2.3.7 *UEFI Capsule Updates:*** The host OS generally provides support for updating the underlying system firmware. This makes the protection of such updates a critical component of the overall system security. The firmware update is passed from the host OS to UEFI in a capsule, which uses digital signatures to validate the integrity and authenticity of firmware update packages [98]. This mechanism establishes a secure framework for delivering firmware updates while safeguarding against unauthorized or malicious firmware installations.

**2.3.8 *BIOS Write Protection:*** Certain UEFI firmware implementations incorporate BIOS write protection. If enabled, it prevents unauthorized modifications to the firmware on the chip [60].

**2.3.9 *User Authentication:*** UEFI incorporates a user authentication mechanism to restrict access to and modification of UEFI settings to authorized individuals [55].

**2.3.10 *Hardware-Based Trusted Execution Environment (TEE):*** Modern CPUs increasingly incorporate built-in TEEs, secure areas of the CPU that ensure confidentiality and integrity of the code and data loaded within it. Leveraging TEEs, UEFI can establish a secure execution environment

that isolates sensitive operations and data from potential threats. TEEs provide hardware-based security features, including secure memory enclaves and encrypted virtualization, which UEFI can harness to enhance its security capabilities. Leveraging TEEs can ensure the confidentiality and integrity of critical system components, protect against memory-based attacks, and strengthen the overall security posture of the system [117].

### 3 UEFI Attacks

#### 3.1 Attacks Observed in the Wild

To identify UEFI-specific threats and vulnerabilities exploited in attacks, we surveyed the academic literature and publicly available technical security reports. We discuss these in the sections below, provide a timeline in Figure 2, and compare them in Table 1.

From reviewing the attacks, we can draw a few conclusions and identify several trends.

- (1) *Evolution of Threat Actors*: In the early phase, (2009-2013), attacks were conducted by well-resourced state actors aiming to carry out covert espionage activities. For example, DarkSeaSkies [30] and DerStrake [111] were tools developed by the Central Intelligence Agency (CIA) for manipulating UEFI. SonicScrewDriver [26] was another CIA tool that enabled attackers to infect UEFI through USB ports. After 2013, the UEFI attack landscape diversified significantly. During this period, the types of threat actors expanded geographically, with a shift in motivations. Financial gain became a significant driver alongside traditional state-sponsored espionage. Recently, UEFI attack tools that were once exclusively available to highly sophisticated actors have become increasingly accessible to a broader audience. These tools are now for sale on various online platforms, such as hacking forums and dark web marketplaces.
- (2) *Increasing Complexity of Attacks*: Modern UEFI malware use more advanced stealth and persistence techniques, as seen in BlackLotus [96] and MoonBounce [102], where attackers employed sophisticated hooking mechanisms to conceal their presence and move to memory locations where detection is extremely difficult. Attackers also employed methods such as modifying UEFI variables, disabling HVCI and PatchGuard [96], and bypassing kernel protection and driver signature enforcement [59, 83].
- (3) *Shift in Attack Objectives*: A shift in objectives is evident, with attacks moving from being purpose-specific to becoming purpose-agnostic. Initial UEFI attacks focused on specific disruptions, whereas recent attacks are directed towards long-term espionage [16], surveillance [94], targeted attacks [14] and data exfiltration [20].
- (4) *Increased Frequency*: The number of detected attacks has risen in recent years.

#### 3.2 Attack PoCs

It is important to note that besides the attacks discovered in the wild, the research community has released proof-of-concepts for additional attacks, many of which are available on public code repositories. For completeness, we address these in the attacks Table 1 as well.



Table 1. UEFI-related Attacks

| Name                       | Year | PoC/<br>Attack | Description                                                                        | Components                                                              | Initial Access                                                             | Initialization and<br>Privilege Escalation                          | Establish Foothold                                     | Maintain Persistence                                                     | Post-Exploitation                                                          | Actions on Objective                                                                 | Ref.          |
|----------------------------|------|----------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------|
| Dark-SeaSkies              | 2009 | Attack         | Malicious implant for Apple MacBook Air.                                           | DarkMatter, SeaPea, and NightSkies.                                     | Insufficient public data available.                                        |                                                                     |                                                        |                                                                          |                                                                            |                                                                                      | [30]          |
| Sonic Screw-driver         | 2012 | Attack         | CIA tool. Exploits Thunderbolt interface. Bypasses firmware password via DMA.      | Insufficient public data available.                                     | Requires physical access. Uses a modified Thunderbolt-to-Ethernet adapter. | Overrides boot process security measures.                           | Embeds within device firmware.                         | Resides in EFI partition. Survives system reboots.                       | Surveillance. Data extraction.                                             | Installs additional espionage tools. Manipulates device operations.                  | [26]          |
| DerStrake                  | 2013 | Attack         | Advanced automated implant for macOS, operates without relying on a physical disk. | Insufficient public data available.                                     | Insufficient public data available.                                        | Maintains presence within the computer's EFI firmware.              | Operates within the disk arbitration process of macOS. | Highly resistant to detection and removal due to EFI firmware embedding. | Often uses network communications through web browsers to evade detection. | Designed for stealthy operations, potentially enabling espionage or data extraction. | [2, 110, 111] |
| DreamBoot                  | 2013 | PoC            | Targets UEFI to attack OS bootloader, requires Secure Boot disabled.               | Hooking functions, malicious payload.                                   | Loaded via modified ISO with EFI PE binary.                                | Modifies bootmgfw.efi and winload.efi, bypasses kernel protections. | Integrates into bootloader and kernel loader.          | Embedded at firmware level.                                              | Disables kernel protections. Executes arbitrary code.                      | Corrupts Windows kernel. Bypasses authentication.                                    | [59, 86]      |
| Thunder-strike             | 2014 | PoC            | Targets macOS via Thunderbolt, implants firmware-level malware.                    | Hooking functions, malicious payload.                                   | Requires physical access. Uses a malicious Thunderbolt device.             | Exploits firmware via Unsigned Option ROMs.                         | Modifies boot ROM to embed malicious code.             | Resides within UEFI firmware.                                            | Can launch further attacks. Deploy malware.                                | Aims for undetectable, persistent root-level access.                                 | [50, 51]      |
| DarkJedi / Thunder-strike2 | 2014 | PoC            | Modifies PEI core firmware, targets UEFI boot script table.                        | Hooking functions, malicious payload.                                   | Through malicious Thunderbolt devices.                                     | Writes to boot flash during PEI phase.                              | Infects motherboard firmware.                          | Embeds in firmware. Survives reinstalls/reboots.                         | Can propagate via infected devices.                                        | Controls firmware/hardware. Leads to data theft.                                     | [52]          |
| VectorEDK                  | 2015 | Attack         | Part of Hacking Team leak, embeds malware in UEFI.                                 | NTFS parser (Ntfs.efi), rkloader.efi, fsbg.efi, scout.exe, soldier.exe. | Likely through physical means or USB devices.                              | Installs DXE driver pre-OS.                                         | Embeds in UEFI, includes NTFS interaction modules.     | Resides in UEFI, survives OS reinstalls.                                 | Can disable security features, spy on users.                               | Espionage. Data theft. System manipulation.                                          | [47, 99]      |
| LightEater                 | 2015 | PoC            | Targets SMM to access physical memory, seeks sensitive data.                       | Insufficient public data available.                                     | Needs physical access or exploit.                                          | Inserts/modifies SMM handlers.                                      | Embeds in SMM.                                         | Remains active across reboots/OS reinstalls.                             | Targets encryption keys, and sensitive data.                               | Data harvesting. potential malware installation.                                     | [66]          |
| PeiBackdoor                | 2015 | PoC            | Enables arbitrary code execution during PEI phase.                                 | Insufficient public data available.                                     | Requires access for firmware modification.                                 | Hooks into early UEFI code.                                         | Integrates within PEI phase.                           | Active from early boot stages.                                           | Alters boot process. Modifies firmware settings.                           | System surveillance. Further malware deployment.                                     | [11]          |
| Thinkpwn                   | 2016 | PoC            | Exploits UEFI vulnerability in Lenovo ThinkPad models.                             | Insufficient public data available.                                     | Requires admin privileges or physical access.                              | Modifies UEFI firmware directly.                                    | Embeds in firmware, loads before OS.                   | Persists through restarts and OS changes.                                | Manipulates system functions. Disables Secure Boot.                        | Espionage. System disruption. Backdoor creation.                                     | [10]          |

Table 1. UEFI-related Attacks

11:10

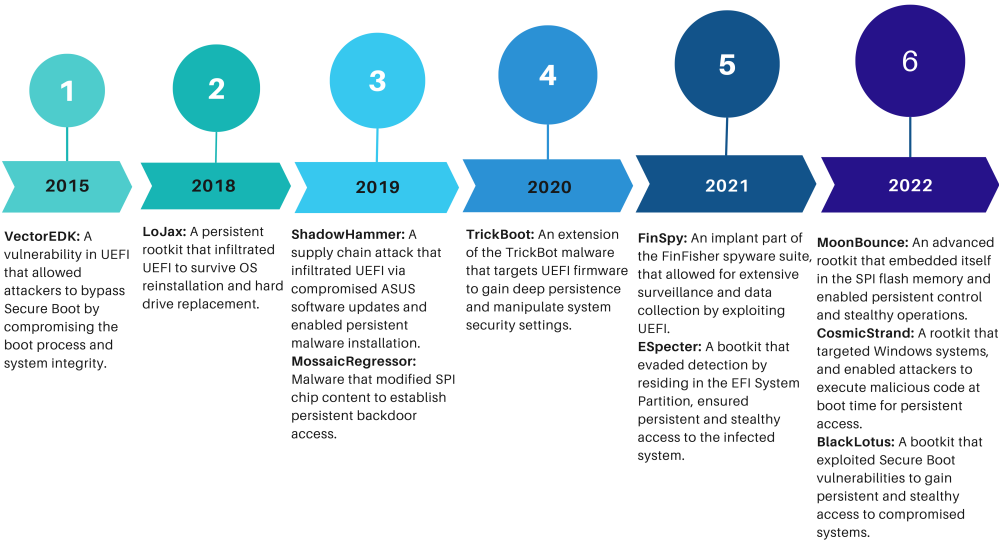
| Name             | Year | PoC/<br>Attack | Description                                                                    | Components                                                                                          | Initial Access                                                         | Initialization and<br>Privilege Escalation                   | Establish Foothold                                   | Maintain Persistence                                            | Post-Exploitation                                                   | Actions on Objective                                                   | Ref.             |
|------------------|------|----------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------|------------------------------------------------------|-----------------------------------------------------------------|---------------------------------------------------------------------|------------------------------------------------------------------------|------------------|
| LoJax            | 2018 | Attack         | Exploits LoJack anti-theft software for UEFI-level persistence.                | SecDxe DXE driver, NTFS driver, malicious payload (Autoche.exe), small agent (Rpcnetp.exe).         | Spear-phishing or compromised websites.                                | Modifies UEFI firmware after gaining admin rights.           | Embeds within UEFI firmware.                         | Survives OS reinstalls and disk replacements.                   | Executes additional malicious payloads at boot.                     | Long-term espionage. Data exfiltration.                                | [49, 73, 88]     |
| Mosaic-Regressor | 2019 | Attack         | Modular cyber-espionage framework targeting diplomatic entities.               | NTFS driver, SMM reset UEFI application, SmmInterfaceBase bootkit, SmmAccessSub persistent dropper. | Likely via phishing or physical access to deploy compromised firmware. | Embeds within UEFI firmware at boot.                         | Executes with each system boot.                      | Remains after hard drive changes/OS reinstalls.                 | Downloads additional modules. Performs espionage activities.        | Espionage. Network compromise. Data theft.                             | [16, 63, 93]     |
| Shadow-Hammer    | 2019 | Attack         | Targeted ASUS Live Update Utility, affecting supply chain.                     | Insufficient public data available.                                                                 | Compromised ASUS utility distributed via official channels.            | Executes a backdoor payload upon update.                     | Uses legitimate certificates to avoid detection.     | Minimal changes to software's file size/signature.              | Executes further malicious payloads.                                | Targeted espionage based on predefined MAC addresses.                  | [14, 29, 62, 92] |
| Trickboot        | 2020 | Attack         | TrickBot module that probes and exploits UEFI/BIOS vulnerabilities.            | PermaDll Module-reconnaissance tool, Firmware Interaction Functions.                                | Spear-phishing or web vulnerabilities.                                 | Checks and exploits UEFI/BIOS settings.                      | Writes payloads directly into UEFI firmware.         | Operates below OS, survives system reinstalls.                  | Disables security measures, downloads payloads.                     | Espionage. Malware delivery. Network compromise.                       | [1, 18, 53, 69]  |
| BootHole         | 2020 | PoC            | Vulnerability in GRUB2 affecting Secure Boot, allows arbitrary code execution. | Insufficient public data available.                                                                 | Requires admin privileges to modify GRUB2 configuration.               | Exploits buffer overflow in GRUB2.                           | Manipulates boot process via GRUB2 config.           | Manipulates bootloader for consistent execution.                | Installs malware. Modifies OS loading.                              | Alters OS loading. Executes arbitrary actions at startup.              | [17]             |
| ESpecter         | 2021 | Attack         | Bypasses Windows driver signature enforcement, persists in ESP.                | Compromised Boot Manager, malicious payloads                                                        | Likely through exploiting vulnerabilities or social engineering.       | Modifies Windows Boot Manager early in the boot process.     | Patches system functions to bypass integrity checks. | Modifies ESP, targeting boot-related files.                     | Deploys kernel-mode driver, sets keyloggers, communicates with C&C. | Executes commands. Downloads malware. Controls compromised machine.    | [28, 83, 95]     |
| FinFisher Finspy | 2021 | Attack         | Surveillance tool that can infect via trojanized installer or UEFI bootloader. | Compromised Boot Manager, Encrypted Payloads.                                                       | Exploits system vulnerabilities or physical access.                    | Embedded at firmware level, operates before OS boots.        | Replaces Windows Boot Manager with a malicious one.  | Resides in UEFI firmware, unaffected by system changes.         | Decrypts/executes payloads, and establishes backdoor.               | Surveillance. Data exfiltration. Further malware delivery.             | [19, 64, 94]     |
| Moon-Bounce      | 2022 | Attack         | Targets SPI flash chip's UEFI firmware, modifies CORE_DXE component.           | Function hooks, malicious payload.                                                                  | Through firmware vulnerabilities or supply chain compromises.          | Manipulates early boot process, controls system pre-OS load. | Resides in non-volatile SPI flash memory.            | Embedded in firmware, unaffected by disk/OS changes.            | Manipulates boot sequence, loads malicious components.              | Network reconnaissance. Data exfiltration. Deploying further payloads. | [70, 102]        |
| Cosmic-Strand    | 2022 | Attack         | Infects UEFI of specific motherboards, used for cyberespionage.                | Function hooks, malicious payload.                                                                  | Speculated physical access or supply chain compromise.                 | Modifies CSMCORE DXE driver, patches entry to redirect.      | Sets up hooks in Windows boot process.               | Alters UEFI, affects even after OS reinstall/drive replacement. | Executes malicious component inside Windows.                        | Targeted data collection. Surveillance. Persistent presence.           | [20, 45, 71, 91] |

Surve, et al.

Table 1. UEFI-related Attacks

| Name        | Year | PoC/<br>Attack | Description                                                    | Components                                                                         | Initial Access                                                                    | Initialization and<br>Privilege Escalation | Establish Foothold                                    | Maintain Persistence                      | Post-Exploitation                                       | Actions on Objective                                           | Ref. |
|-------------|------|----------------|----------------------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------|-------------------------------------------------------|-------------------------------------------|---------------------------------------------------------|----------------------------------------------------------------|------|
| Black-Lotus | 2022 | Attack         | First UEFI attack to bypass Secure Boot, sold on the dark web. | Malicious bootloader, malicious BCD, legitimate but vulnerable binary executables. | Installs via legitimate but vulnerable Windows binaries, possibly online/offline. | Disables Secure Boot using CVE-2022-21894. | Sets up legitimate shim binary as default bootloader. | Deploys kernel driver to prevent removal. | Deploys user-mode components, executes kernel payloads. | Full control over OS boot process. Deploys arbitrary payloads. | [96] |

Fig. 2. UEFI Attack Timeline



4 UEFI MITRE ATT&CK-like Mapping

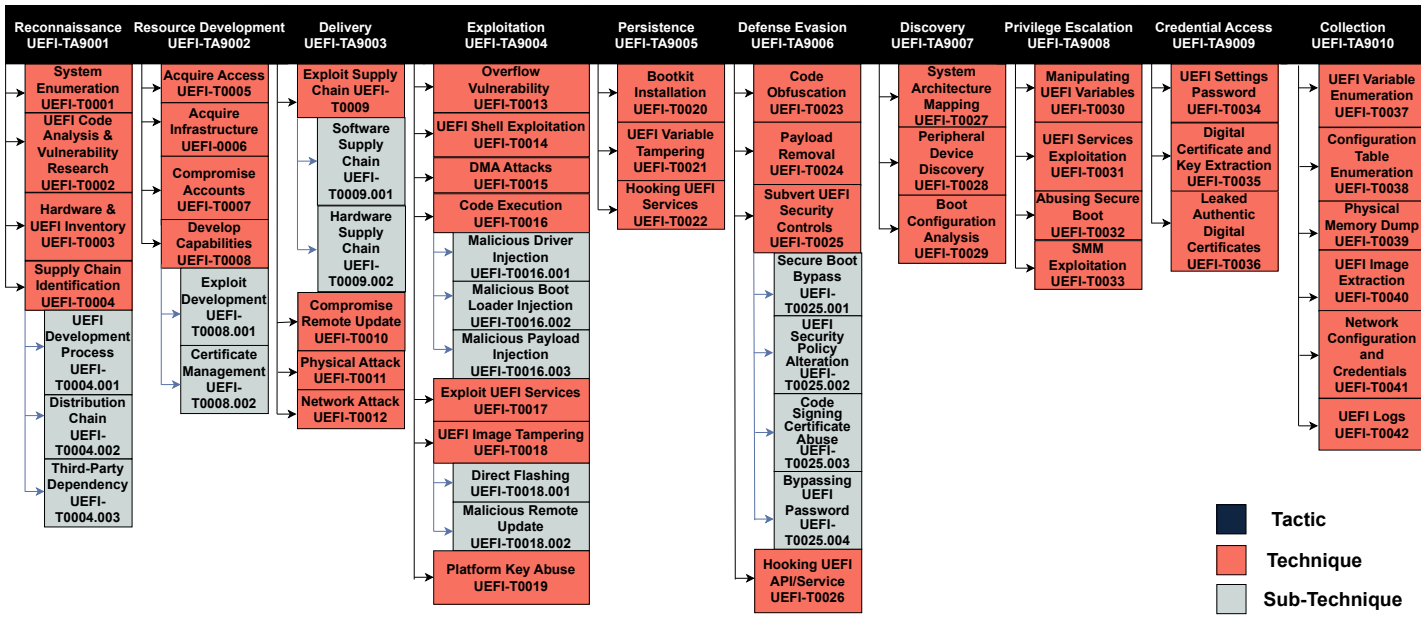
During the investigation of UEFI attacks (as summarized in Table 1, we concluded that the various attack patterns can be broken down into basic elements and documented in a similar way to other domains. Consequently, we created a MITRE-like mapping of the tactics and techniques used in UEFI-related attacks. This mapping can be applied to both old and new attacks, categorizing them by TTPs (Tactics, Techniques, and Procedures) and stages, providing a common language for security professionals and researchers. We acknowledge that this mapping may not be exhaustive, and we invite the UEFI security research community to adopt it and expand it. In creating the framework, we intentionally excluded non-UEFI perspectives, even if they are relevant to specific UEFI attack campaigns. For example, network scanning might be part of a UEFI-related campaign; however, since it is not specific to UEFI, it is not included in the framework. This decision is based on the understanding that cybersecurity attacks are often complex, involving a variety of tools, techniques, and capabilities that may extend beyond a specific sub-domain. Therefore, we focused on UEFI-specific components of the attack, which we outline here, while excluding non-UEFI-specific elements that are covered in existing literature and other frameworks. To demonstrate the practical utility of such a mapping, we use it as a basis for conducting a comprehensive risk analysis of UEFI-related security concerns in [75] where we map existing security countermeasures against each technique. A brief description of the attack techniques follows. For a detailed description of each technique, we encourage readers to refer to the respective references in Section 2.

Table 2. UEFI Techniques and Sub-Techniques

| Tactic                                | Technique                                                  | Sub-Technique                                    | Reference                          |
|---------------------------------------|------------------------------------------------------------|--------------------------------------------------|------------------------------------|
| Reconnaissance<br>(UEFI-TA9001)       | System Enumeration (UEFI-T0001)                            | -                                                | [96], [1], [53], [14]              |
|                                       | UEFI Code Analysis and Vulnerability Research (UEFI-T0002) | -                                                | [18], [88], [96]                   |
|                                       | Hardware and UEFI Inventory (UEFI-T0003)                   | -                                                | [18], [53]                         |
|                                       | Supply Chain Identification (UEFI-T0004)                   | UEFI Development Process (UEFI-T0004.001)        | [14]                               |
|                                       |                                                            | Distribution Chain (UEFI-T0004.002)              | [62], [92]                         |
|                                       |                                                            | Third-Party Dependency (UEFI-T0004.003)          | [14], [3]                          |
| Resource Development<br>(UEFI-TA9002) | Acquire Access (UEFI-T0005)                                |                                                  | [18], [14]                         |
|                                       | Acquire Infrastructure (UEFI-T0006)                        |                                                  | [63], [73]                         |
|                                       | Compromise Accounts (UEFI-T0007)                           |                                                  | [14]                               |
|                                       | Develop Capabilities (UEFI-T0008)                          | Exploit Development (UEFI-T0008.001)             | [4], [73]                          |
|                                       |                                                            | Certificate Management (UEFI-T0008.002)          | [14], [29]                         |
| Delivery<br>(UEFI-TA9003)             | Exploit Supply Chain (UEFI-T0009)                          | Software Supply Chain (UEFI-T0009.001)           | [62], [92], [20], [88]             |
|                                       |                                                            | Hardware Supply Chain (UEFI-T0009.002)           | [20], [70]                         |
|                                       | Compromise Remote Update (UEFI-T0010)                      |                                                  | [88], [62]                         |
|                                       | Physical Attacks (UEFI-T0011)                              |                                                  | [26], [50], [51], [99]             |
|                                       | Network Attacks (UEFI-T0012)                               |                                                  | [53], [70], [111]                  |
| Exploitation<br>(UEFI-TA9004)         | Overflow Vulnerability (UEFI-T0013)                        |                                                  | [17], [96]                         |
|                                       | UEFI Shell Exploitation (UEFI-T0014)                       |                                                  | [104]                              |
|                                       | DMA Attacks (UEFI-T0015)                                   |                                                  | [115], [26]                        |
|                                       | Code Execution Attacks (UEFI-T0016)                        | Malicious Driver Injection (UEFI-T0016.001)      | [11], [73], [70]                   |
|                                       |                                                            | Malicious Boot Loader Injection (UEFI-T0016.002) | [59], [96], [83]                   |
|                                       |                                                            | Malicious Payload Injection (UEFI-T0016.003)     | [73], [63], [99]                   |
|                                       | Exploit UEFI Services (UEFI-T0017)                         |                                                  | [106]                              |
|                                       | UEFI Image Tampering (UEFI-T0018)                          | Direct Flashing (UEFI-T0018.001)                 | [70], [20], [99]                   |
|                                       |                                                            | Malicious Remote Update (UEFI-T0018.002)         | [29], [88]                         |
| Persistence<br>(UEFI-TA9005)          | Platform Key Abuse (UEFI-T0019)                            |                                                  | [96]                               |
|                                       | Bootkit Installation (UEFI-T0020)                          |                                                  | [99], [88], [70], [20], [95], [96] |
|                                       | UEFI Variable Tampering (UEFI-T0021)                       |                                                  | [47], [16], [96]                   |
|                                       | Hooking UEFI Services (UEFI-T0022)                         |                                                  | [70], [71], [63], [95]             |
| Defense Evasion<br>(UEFI-TA9006)      | Code Obfuscation (UEFI-T0023)                              |                                                  | [64], [96]                         |
|                                       | Payload Removal (UEFI-T0024)                               |                                                  | [73]                               |
|                                       | Subvert UEFI Security Controls (UEFI-T0025)                | Secure Boot Bypass (UEFI-T0025.001)              | [96], [94]                         |
|                                       |                                                            | UEFI Security Policy Alteration (UEFI-T0025.002) | [96]                               |
|                                       |                                                            | Code Signing Certificate Abuse (UEFI-T0025.003)  | [29]                               |
|                                       |                                                            | Bypassing UEFI Password (UEFI-T0025.004)         | [24]                               |
| Discovery<br>(UEFI-TA9007)            | Hooking UEFI API/Services (UEFI-T0026)                     |                                                  | [70], [71], [83]                   |
|                                       | System Architecture Mapping (UEFI-T0027)                   |                                                  | [53]                               |
|                                       | Peripheral Device Discovery (UEFI-T0028)                   |                                                  | [53]                               |
|                                       | Boot Configuration Analysis (UEFI-T0029)                   |                                                  | [96]                               |
| Privilege Escalation<br>(UEFI-TA9008) | Manipulating UEFI Variables (UEFI-T0030)                   |                                                  | [96], [106], [61]                  |
|                                       | UEFI Services Exploitation (UEFI-T0031)                    |                                                  | [61]                               |
|                                       | Abusing Secure Boot (UEFI-T0032)                           |                                                  | [96], [81]                         |
|                                       | SMM Exploitation (UEFI-T0033)                              |                                                  | [66], [10], [98]                   |

| Tactic                             | Technique                                           | Sub-Technique | Reference         |
|------------------------------------|-----------------------------------------------------|---------------|-------------------|
| Credential Access<br>(UEFI-TA9009) | UEFI Setting Password (UEFI-T0034)                  |               | [78]              |
|                                    | Leaked Authentic Digital Certificates (UEFI-T0035)  |               | [29], [14]        |
|                                    | Digital Certificate and Key Extraction (UEFI-T0036) |               | [14]              |
| Collection<br>(UEFI-TA9010)        | UEFI Variable Enumeration (UEFI-T0037)              |               | [90], [106]       |
|                                    | Configuration Table Enumeration (UEFI-T0038)        |               | [106]             |
|                                    | Physical Memory Dump (UEFI-T0039)                   |               | [66]              |
|                                    | UEFI Image Extraction (UEFI-T0040)                  |               | [5]               |
|                                    | Network Configuration and Credentials (UEFI-T0041)  |               | [106]             |
|                                    | UEFI Logs (UEFI-T0042)                              |               | [85], [106], [90] |

Fig. 3. Proposed UEFI MITRE ATT&CK-like framework



#### 4.1 Taxonomy Construction Methodology

The taxonomy was derived through a systematic, evidence-driven process grounded in the attack corpus described in Section 3. The construction process followed four sequential steps.

**Step 1: Systematic Attack Review.** All publicly documented UEFI-related attacks and proof-of-concept implementations available at the time of study were reviewed systematically, as summarized in Table 1. For each attack, the following dimensions were extracted: the initial access vector, the exploitation mechanism, the persistence technique, the defense evasion approach, and the post-exploitation objectives. This extraction was performed directly from primary sources, including academic publications, security vendor reports, and publicly available technical analyses, ensuring that the taxonomy reflects observed real-world behavior rather than theoretical attack models.

**Step 2: Pattern Identification and Grouping.** Extracted attack steps were analyzed to identify recurring patterns across multiple attacks. Steps exhibiting similar objectives and mechanisms were grouped into candidate techniques. Candidate techniques that shared a common adversarial goal at a higher level of abstraction were further grouped into candidate tactics. This bottom-up grouping process was guided by the MITRE ATT&CK framework [75] as a structural reference, ensuring compatibility with existing security tooling and terminology while remaining grounded in UEFI-specific evidence.

**Step 3: Inclusion and Exclusion Criteria.** A tactic or technique was included in the taxonomy if it was supported by the reviewed UEFI attack corpus and represented a behavior that was meaningfully tied to the UEFI or firmware attack surface. Behaviors that may appear in UEFI-related campaigns but are not themselves firmware-specific were not modeled as standalone taxonomy elements unless they played a distinct role in enabling, targeting, or exploiting the UEFI environment. This design choice explains why the proposed framework contains 10 tactics rather than mirroring all 14 tactics in the full MITRE ATT&CK enterprise framework.

**Step 4: Iterative Review and Refinement.** The resulting taxonomy was subjected to iterative review against the full attack corpus to verify that the documented attacks could be consistently represented using the proposed tactics and techniques. Where initial groupings were found to be ambiguous, technique boundaries were refined to improve consistency and preserve a clear UEFI-specific focus. The taxonomy was further reviewed against the UEFI specification and related security literature to verify that the defined techniques corresponded to distinct, technically meaningful stages of UEFI exploitation. We acknowledge that the taxonomy reflects the state of publicly documented attacks at the time of study and may not capture attack techniques that have not yet been publicly disclosed. We therefore present it as a living framework and invite the UEFI security research community to adopt, validate, and extend it as the threat landscape evolves.

#### 4.2 Mapping UEFI Attacks to MITRE-like TTPs (Tactics, Techniques and Procedures)

In this section, we discuss the TTPs within the UEFI MITRE ATT&CK-like framework. The tactic and technique identifiers used in this framework follow a numbering scheme beginning at 9001 (for tactics) and T0001 (for techniques) to avoid conflicts with existing MITRE ATT&CK enterprise identifiers, which use four-digit codes in overlapping ranges. This ensures that the UEFI-specific framework can be used alongside existing MITRE frameworks without identifier collision.

- **Reconnaissance (UEFI-TA9001):** During reconnaissance, a threat actor gathers information about the target. Within the context of UEFI, this process can include the following:
  - **System Enumeration (UEFI-T0001):** Attackers aim to identify the target system's UEFI firmware version, vendor, and specific model [53]. This information helps them understand the firmware's features, potential vulnerabilities, and available exploits.



- **Firmware Code Analysis and Vulnerability Research (UEFI-T0002):** Threat actors analyze UEFI firmware code to identify potential security weaknesses, such as known issues in outdated firmware versions [14], known vulnerabilities [96], or misconfigurations [88].
- **Hardware and Firmware Inventory (UEFI-T0003):** Threat actors gather information about the hardware components integrated into the system, such as the motherboard, chipset, storage devices, and network adapters. They also identify the firmware versions associated with these components, as outdated drivers can be potential entry points for exploitation [1].
- **Supply Chain Identification (UEFI-T0004):** Threat actors analyze the supply chain for potential security weaknesses at various stages, from development to distribution. In this context, the threat actors do any of the following:
  - \* **Firmware Development Process (UEFI-T0004.001):** Gather information about the firmware development and integration process, tools, methodologies, and security practices employed by the IBV/ODM/OEM.
  - \* **Distribution Chain (UEFI-T0004.002):** Analyze the security measures deployed by vendors during manufacturing, development, transportation, storage, and installation of the UEFI.
  - \* **Third-Party Dependencies (UEFI-T0004.003):** Analyze the dependencies and third-party hardware/software used in the system to identify vulnerabilities in those components that can be exploited to compromise the integrity of the UEFI.
- **Resource Development (UEFI-TA9002):** During the resource development stage, threat actors use the information obtained during reconnaissance to craft the tools and means needed to achieve their campaign goals. In this context, they do the following:
  - **Acquire Access (UEFI-T0005):** The adversary purchases access to the target via access brokers [18].
  - **Acquire Infrastructure (UEFI-T0006):** Adversaries establish infrastructure such as C&C servers to deliver payloads, receive instructions, or maintain persistence within targeted systems [63, 73].
  - **Compromise Accounts (UEFI-T0007):** Attackers compromise accounts with privileged access to jeopardize the security and integrity of UEFI [14].
  - **Develop Capabilities (UEFI-T0008):** Attackers develop capabilities, such as exploits, to carry out malicious activities. Those capabilities include the following sub-techniques:
    - \* **Exploit Development (UEFI-T0008.001):** Attackers create or obtain exploit code to target specific UEFI vulnerabilities, such as customizing a UEFI DXE driver [4].
    - \* **Certificate Management (UEFI-T0008.002):** Attackers acquire authentic digital certificates from vendors or generate their own to sign malicious UEFI code [14].
- **Delivery (UEFI-TA9003)** During this stage, the threat actor attempts to introduce, transmit, or position the exploit or malicious component into the UEFI or pre-OS attack surface. In this taxonomy, the Delivery tactic is limited to UEFI-specific exploit delivery paths, such as supply-chain compromise, remote firmware update paths, physical access, or network-reachable firmware interfaces. More general command-and-control activity is outside the scope of this UEFI-specific framework and is therefore not modeled as a separate tactic here. This includes:
  - **Exploit Supply Chain (UEFI-T0009):** Attackers compromise supply-chain components of UEFI systems to deliver a malicious payload to the target system. This can be accomplished using the following sub-techniques:

- \* **Software Supply Chain (UEFI-T0009.001):** Attackers hijack the UEFI firmware development and distribution mechanism [20, 62].
- \* **Hardware Supply Chain (UEFI-T0009.002):** Attackers infiltrate the hardware manufacturing process to compromise the hardware components such as the motherboard (SPI chip) and 3rd party components, ensuring their malicious code (e.g., DXE driver) is executed when the hardware is initiated.
- **Compromise Remote Update (UEFI-T0010):** Attackers attempt to exploit vulnerabilities in the firmware update mechanism provided by the UEFI. An attacker can trick the system into accepting and applying a malicious update, delivering their payload into the UEFI [92].
- **Physical Access Attack (UEFI-T0011):** Attackers gain unauthorized physical access to the target system to modify the UEFI firmware by tampering with the SPI flash chip or other storage devices where the firmware is stored. Such an attack can typically be carried out by an insider threat [26, 50, 99].
- **Network Access Attacks (UEFI-T0012):** The threat actor initiates communication with a network-connected target to gain access [53, 70, 111].
- **Exploitation (UEFI-TA9004):** During this stage, the threat actor attempts to exploit the target system by running malicious code or using native capabilities (e.g., "living off the land" attacks).
  - **Overflow Vulnerability (UEFI-T0013):** Vulnerabilities can occur when inputs to UEFI system components are not properly validated, or when buffer limits are not enforced, causing data to overflow into adjacent memory regions. Attackers gain unauthorized access to such memory regions and misuse the sensitive information gathered from those regions [17].
  - **UEFI Shell Exploitation (UEFI-T0014):** If the UEFI implementation supports a shell environment, it can be exploited by the attackers to execute arbitrary UEFI shell scripts or commands [104].
  - **Direct Memory Access (DMA) Attacks (UEFI-T0015):** Attackers exploit the DMA capabilities (the ability to access memory directly) of peripheral devices such as network interface cards, graphics cards, and storage controllers to compromise UEFI firmware or the underlying operating system. This leads to malicious code injection and privilege escalation [115].
  - **Code Execution (UEFI-T0016):** Code injection gives attackers the ability to execute arbitrary code on the system by exploiting vulnerabilities in UEFI firmware or associated components, using the following sub-techniques:
    - \* **Malicious Driver Injection (UEFI-T0016.001):** Attackers can inject and execute a malicious driver during the DXE phase of booting. This allows attackers to gain control over the system very early, potentially compromising UEFI security and integrity [11, 51, 73].
    - \* **Malicious Boot Loader Injection (UEFI-T0016.002):** Attackers modify or replace the legitimate bootloader with a malicious one. The bootloader loads the operating system (OS) into memory and initiates system startup. Injecting a malicious bootloader can give attackers control over the boot process [17, 64, 86, 96].
    - \* **Malicious Payload Injection (UEFI-T0016.003):** Attackers inject and execute malicious code within the UEFI firmware environment through custom code or firmware updates [17, 50, 59].
  - **Exploit UEFI Services (UEFI-T0017):** Threat actors exploit various UEFI services to carry out attacks, such as EFI\_BOOT\_SERVICES to exploit various boot-related

- functions (e.g., loading bootloaders and operating systems), `EFI_VARIABLE_SERVICES` to modify UEFI variables (potentially altering system behavior or disabling security features), `EFI_FILE_SERVICES` to manipulate or replace critical UEFI firmware files, `EFI_TEXT_INPUT_PROTOCOL` to inject malicious commands or bypass security measures, and `EFI_NETWORK_INTERFACE_IDENTIFIER_PROTOCOL` to launch remote UEFI attacks, such as firmware updates over the network [106].
- **UEFI Image Tampering (UEFI-T0018):** Tampering with the firmware image is used to establish persistence, bypass UEFI security features, create hidden backdoors within the firmware, provide unauthorized access to the system, or compromise its security. The following sub-techniques are observed at this stage:
    - \* **Physical Access (UEFI-T0018.001):** Attackers replace the firmware with a trojanized version by writing the malicious firmware directly to the SPI chip, typically using specialized software or tools provided by the hardware manufacturer [70].
    - \* **Malicious Remote Update (UEFI-T0018.002):** Attackers exploit vulnerabilities in the UEFI remote update process to inject malicious firmware updates by manipulating firmware update servers or using social engineering tactics to trick users into applying the update [29, 63].
  - **Platform Key Abuse (UEFI-T0019):** Attackers sign malicious UEFI firmware, bootloaders, or drivers with a compromised platform key, making them appear legitimate and trusted by the UEFI.
  - **Persistence (UEFI-TA9005)** These techniques ensure that the threat actor maintains control of the infected system, even after a reboot. A threat actor accomplishes persistence in several ways:
    - **Bootkit Installation (UEFI-T0020):** Establish persistence on a compromised system by embedding malicious code within the boot process [99], ensuring it executes each time the system starts up.
    - **UEFI Variable Tampering (UEFI-T0021):** UEFI variable tampering ensures that the attacker's malicious configurations or code remain effective across multiple system reboots. Attacker does so by changing the variables containing individual boot entries and defining the paths to bootloader files, the order in which boot devices are accessed during the boot process, PK and KEK variables, etc [16, 47].
    - **Hooking UEFI Services (UEFI-T0022):** UEFI runtime services provide various functions accessible even after the operating system has been booted. Attackers modify or "hook" these services by injecting malicious code or modifying existing service handlers. Handlers such as `GetVariable` and `SetVariable` are targeted to manipulate UEFI variables related to boot order, firmware settings, or security policies [70]. `LocateProtocol` and `LocateHandle` are used to access specific firmware interfaces or to locate functions or resources. `InstallProtocolInterface` is exploited to introduce malicious code into protocols or interfaces. `AllocatePages` and `FreePages` allocate memory for code or data structures, ensuring persistence, while `RaiseTPL` is used to gain higher privileges by raising the Task Priority Level [63, 95, 106].
  - **Defense Evasion (UEFI-TA9006)**
    - **Code Obfuscation (UEFI-T0023):** Attackers obfuscate malicious UEFI code to make it more difficult for defenders to analyze and identify its purpose. Obfuscation techniques can include encryption, encoding, or packing the code [64, 96].
    - **Payload Removal (UEFI-T0024):** Attackers design their malware to remove malicious files after infecting the target [73].

- **Subvert UEFI Security Controls (UEFI-T0025):** UEFI firmware includes various security mechanisms designed to protect the system against unauthorized access, malware, and other threats [95, 96]. The attackers exploit vulnerabilities to bypass or undermine the security features such as secure boot, capsule update, and code-signing certificate using the following techniques:
  - \* **Secure Boot Bypass (UEFI-T0025.001):** Secure Boot ensures that only trusted and signed firmware, bootloaders, and drivers are executed during the boot process. Vulnerabilities in Secure Boot implementation enables threat actors to load and execute unsigned or malicious code, bypassing the integrity checks performed by Secure Boot [94, 96].
  - \* **UEFI Security Policy Alteration (UEFI-T0025.002):** Attackers perform unauthorized modifications or deletions of the UEFI security policies and settings related to Secure Boot, UEFI Secure Flash, platform key management, etc., to escalate privileges, establish persistence, or create a backdoor [96].
  - \* **Code Signing Certificate Abuse (UEFI-T0025.003):** Code signing verifies the authenticity and integrity of software by attaching a digital signature. With a stolen or fraudulent code signing certificate, attackers can sign malicious UEFI firmware, drivers, or other components that are executed during the boot process or firmware update and are considered legitimate due to their valid digital signatures [29].
  - \* **Bypassing UEFI Password (UEFI-T0025.004):** Some systems allow users to set passwords to protect UEFI settings. Attackers use multiple techniques, such as dictionary/brute force attacks, keylogging, or social engineering, to bypass or recover these passwords, gaining administrative access and control over firmware settings [24].
- **Hooking UEFI API/Service (UEFI-T0026):** Attackers hook UEFI services or redirect UEFI API calls to their own code. This allows them to intercept and manipulate API calls and evade security controls [70, 71, 83].
- **Discovery (UEFI-TA9007):**
  - **System Architecture Mapping (UEFI-T0027):** Attackers aim to identify the architecture and components of the target system, including the motherboard, chipset, processors, memory modules, storage devices, and network interfaces. This information helps them understand the system’s capabilities, potential attack vectors, and areas of focus for post-exploitation [53].
  - **Peripheral Device Discovery (UEFI-T0028):** Attackers identify peripheral devices connected to the target system, such as USB devices, network adapters, or storage devices. This helps them understand the potential post-exploitation avenues associated with these devices.
  - **Boot Configuration Analysis (UEFI-T0029):** Attackers explore the boot configuration settings in UEFI firmware, such as the boot order, boot options, and boot manager entries. This involves studying the boot process and associated components to identify weak points or misconfigurations. This information can be useful for post-exploitation manipulation of the boot process or alteration of boot parameters [96].
- **Privilege Escalation (UEFI-TA9008):**
  - **Manipulating UEFI Variables (UEFI-T0030):** UEFI variables store system settings and configurations, including the security policy. Attackers manipulate these variables to change security settings, boot order, or other critical configurations, thereby escalating their privileges within the firmware environment [106].

- **UEFI Service Exploitation (UEFI-T0031):** Manipulating UEFI services, such as UEFI variable services, UEFI handles, and task-level priority, allows attackers to gain unauthorized control over the boot process and system configuration [61]. At this elevated access level, attackers can execute their code without any restrictions.
- **Abusing Secure Boot (UEFI-T0032):** By exploiting vulnerabilities in secure boot, attackers can inject unsigned code at the firmware level. This results in privilege escalation at the OS level, providing attackers with root-level access [81].
- **SMM Exploitation (UEFI-T0033):** Attackers exploit vulnerabilities in SMM code to gain low-level control of the system, effectively escalating their privileges within the firmware [98].
- **Credential Access (UEFI-TA9009):**
  - **UEFI Settings Password (UEFI-T0034):** UEFI implementations allow users to set passwords to protect UEFI settings. Attackers attempt to extract or bypass these passwords to gain unauthorized access to UEFI settings and credentials [78].
  - **Leaked Authentic Digital Certificates (UEFI-T0035):** Leaked digital certificates, particularly those used for code signing or secure communication, can be exploited by attackers. If attackers gain access to these certificates, they can sign malicious firmware or bootloader components, making them appear legitimate during the Secure Boot verification process [29].
  - **Digital Certificate and Key Extraction (UEFI-T0036):** If digital certificates or encryption keys are stored in UEFI firmware for secure communication or firmware updates, attackers attempt to extract these credentials for later use [14].
- **Collection (UEFI-TA9010):**
  - **UEFI Variable Enumeration (UEFI-T0037):** Attackers enumerate UEFI variables to gather information about system settings, configurations, or stored data [90].
  - **Configuration Table Enumeration (UEFI-T0038):** UEFI firmware uses configuration tables to store information about system hardware and resources. Attackers enumerate these tables to gather details about the system’s hardware components and configurations [106].
  - **Physical Memory Dump (UEFI-T0039):** Attackers with access to a system’s physical memory perform memory dumps to collect data stored in RAM. This data can include UEFI variables, SMM data, or other sensitive information present in memory during the boot process.
  - **Firmware Image Extraction (UEFI-T0040):** Attackers attempt to extract the UEFI firmware image from the system for analysis or reverse engineering.
  - **Network Configuration and Credentials (UEFI-T0041):** UEFI firmware stores network-related configurations and credentials for services like PXE (Preboot Execution Environment) boot or remote management. Attackers extract these credentials for unauthorized network access [106].
  - **UEFI Logs (UEFI-T0042):** Attackers can retrieve measured boot logs (if enabled) or records related to UEFI updates to gather information about previous updates, vulnerabilities, or changes in the firmware [85].

## 5 Analysis of Existing Countermeasures

As the threat landscape evolves, the importance of robust countermeasures to safeguard UEFI systems is growing. Based on the TTP mapping (see Section 4), we identified existing countermeasures to defend UEFI against these threats and, where relevant, we identified gaps and propose additional countermeasures. Consistent with the literature, we have broadly classified the countermeasures

into categories: prevention, detection, mitigation, and remediation. In line with NIST CSF [76], we have also addressed the "identification" category. Preventive, detective, mitigation, and remediation controls are designed to raise the cost of an attack, while identification controls are designed to understand which organizational assets need protection. However, it is important to remember that the level of abstraction in this paper differs: instead of focusing on the organizational security level (as per NIST CSF), we address the specifics of UEFI. For completeness, and inspired by this category, we mapped the various components of UEFI, as shown in the background section. The countermeasures in the other categories (e.g., prevention, detection, mitigation, and remediation) are summarized in an MITRE-like framework in Figure 4. A brief description of the countermeasures follows; for a detailed description of each countermeasure, refer to the references in Table 3.

Table 3. Countermeasures for UEFI Security

| Category   | Sub-type                                            | List of Countermeasures                                     | Reference              |
|------------|-----------------------------------------------------|-------------------------------------------------------------|------------------------|
| Preventive | UEFI Hardening PM-01                                | Use Secure Boot                                             | [78], [81], [43]       |
|            |                                                     | Implement UEFI image signing                                | [78], [81]             |
|            |                                                     | BIOS integrity check                                        | [78], [82]             |
|            |                                                     | Set flash descriptor region to Read-Only                    | [65], [24]             |
|            |                                                     | Disable unnecessary SMM features                            | [65], [98]             |
|            |                                                     | Encrypt sensitive memory areas                              | [35]                   |
|            |                                                     | Memory access control                                       | [115], [35]            |
|            |                                                     | Boot order verification                                     | [78], [46]             |
|            |                                                     | Memory integrity verification                               | [78], [35]             |
|            |                                                     | Regularly update DBX database                               | [15], [74]             |
|            |                                                     | Lock down SPI flash memory access                           | [65], [24]             |
|            |                                                     | Full-disk encryption                                        | [85]                   |
|            |                                                     | Disable legacy protocols and services                       | [78]                   |
| Preventive | Supply Chain Security PM-02                         | Assess and audit vendors in the UEFI supply chain           | [8], [34]              |
|            |                                                     | Use secure, traceable supply chain method                   | [3], [34]              |
|            |                                                     | Use components from trusted sources only                    | [8], [43]              |
|            |                                                     | 3rd party component integrity check                         | [44], [8]              |
|            |                                                     | Incorporate SBOM into SSDLC                                 | [34], [3], [42]        |
| Preventive | Authentication, Authorization, Access Control PM-03 | Key management policy enforcement                           | [36], [40]             |
|            |                                                     | Implement robust Authentication Protocols                   | [103], [33]            |
|            |                                                     | Access control to UEFI settings and features                | [78], [33]             |
|            |                                                     | Disable USB Boot                                            | [78]                   |
|            |                                                     | Ensure Physical Security of Ports                           | [78], [23]             |
| Preventive | Update Patch Management PM-04                       | Disable UEFI Version Rollback                               | [31]                   |
|            |                                                     | Patch and Update the UEFI Regularly                         | [78], [44]             |
|            |                                                     | Implement UEFI Version Control                              | [44], [97]             |
|            |                                                     | Implement vulnerability management program                  | [44], [87]             |
|            |                                                     | Verify integrity of update tools                            | [78], [39]             |
|            |                                                     | Check signature before applying UEFI update                 | [78], [81]             |
|            |                                                     | MFA for update process                                      | [103], [27]            |
| Preventive | Execution Control PM-05                             | Software execution policy                                   | [36], [106]            |
|            |                                                     | Run only signed drivers                                     | [80], [38]             |
|            |                                                     | Firmware integrity                                          | [37], [40], [78], [41] |
| Detective  | System Behavior Monitoring DM-01                    | Monitor SMM access and activities                           | [115], [98]            |
|            |                                                     | Monitor system during loading of a driver                   | [78], [65]             |
|            |                                                     | Scan for unauthorized memory access patterns                | [115], [35]            |
|            |                                                     | Monitor for memory patching attempts                        | [115], [65]            |
|            |                                                     | Monitor DMA access                                          | [115], [22]            |
|            |                                                     | Monitor unexpected Write operations to the flash descriptor | [65], [24]             |

Continued on next page

Table 3 – Continued from previous page

| Type        | Sub-type                    | List of Countermeasures                                                                                        | Reference                              |
|-------------|-----------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------|
|             |                             | Monitor UEFI logs                                                                                              | [78], [90]                             |
| Detective   | Anomaly Detection DM-02     | Monitor for unusual activities<br>Conduct UEFI static analysis                                                 | [68], [114]<br>[7], [68], [39]         |
| Mitigation  | Incident Response MM-01     | Investigation of Suspicious Activities<br>Analyze security feature bypass<br>Analyze supply chain impact       | [39], [12]<br>[60], [39]<br>[8], [34]  |
| Mitigation  | Isolation MM-02             | Isolate the affected system<br>Disable compromised update paths<br>Isolate and disable compromise SMM features | [12], [39]<br>[12], [78]<br>[98], [60] |
| Mitigation  | Credential Management MM-03 | Revoke compromised credentials<br>Revoke compromised cryptographic keys                                        | [13], [25], [74]<br>[40], [32], [74]   |
| Remediation | UEFI Restoration RM-01      | Reflash UEFI<br>Reissue cryptographic keys<br>Restore credentials                                              | [44], [12]<br>[40], [32]<br>[12], [78] |

5.1 Preventive Measures

Preventive countermeasures aim to prevent threat actors from posing a risk to UEFI.

**UEFI Hardening (PM-01):** Implementing this countermeasure involves several steps, beginning with secure boot, which ensures the integrity of firmware components by preventing the execution of unsigned code (e.g., UEFI drivers and applications) during boot [78]. UEFI image signing enhances security by authenticating firmware components and safeguarding against unauthorized modifications [78]. Cryptographic checks of UEFI updates ensure that only verified updates are applied, maintaining firmware security [81]. Additionally, setting the flash descriptor region to read-only [65] and disabling unnecessary SMM features [98] reduce the risk of unauthorized modifications. Encrypting sensitive memory areas further fortifies the system against unauthorized access to data and settings [43]. Checks like boot order verification are useful in preventing a variety of attacks [46]. Memory integrity verification and memory access control implies implementing mechanisms which prevent unauthorised access to UEFI related structures during boot and run-time. Full-disk encryption (FDE) is a key method for protecting data by encrypting the entire disk, preventing unauthorized access to the ESP partition, which stores UEFI settings [85].

**Supply Chain Security (PM-02):** Ensuring the security of the UEFI supply chain is crucial. Protecting the UEFI supply chain involves thorough assessment and auditing of vendors to ensure compliance with security standards and UEFI specifications, using secure and traceable software and hardware development and delivery methods [3], sourcing software and hardware from trusted sources, performing integrity checks on third-party software and hardware components, and incorporating a software bill of materials (SBOM) into the secure software development life cycle (SSDLC) to enhance transparency [8, 34, 43].

**Access, Authentication & Authorization Control (PM-03):** This includes restricting access to UEFI settings with password protection to prevent unauthorized configuration modifications. Disabling the ability to boot from USB devices helps protect against boot-level malware. Ensuring the physical security of ports prevents direct access attacks, such as booting from an external device or using hardware keyloggers [33]. Enforcing secure key management policies and implementing robust authentication protocols, are crucial for maintaining the security of UEFI systems [36, 103].

**Update & Patch Management (PM-04):** Effective management includes disabling UEFI version rollback to prevent the use of older, vulnerable firmware versions [31], regularly deploying patches to address security issues [78], managing firmware versions by monitoring fleet-wide UEFI versioning

and tracking changes with regular assessments, requiring MFA for updates, and implementing a comprehensive vulnerability management program for tracking and remediating vulnerabilities [39, 44].

**Execution Control (PM-05):** This countermeasure involves several controls that restrict code execution in a UEFI setting. For instance, code execution policy enforces rules that allow only pre-approved UEFI-related code (e.g., drivers and applications) to run [36]. Signed driver enforcement ensures that only authenticated drivers are executed in the UEFI environment [80]. Cryptographic methods like digital signatures, hash functions, and certificates should also be used to ensure UEFI firmware integrity [37, 40, 106].

## 5.2 Detective Measures

Detective measures focus on monitoring system behavior to identify potential security breaches or unauthorized activities that may indicate an active attack.

**System Behavior Monitoring (DM-01):** Monitoring activities include checking SMM access, conducting regular UEFI log audits, performing BIOS integrity checks, scanning for unauthorized UEFI/SMM memory access patterns, monitoring UEFI/SMM memory patching attempts, detecting direct memory access (DMA) operations [115], and observing unexpected write operations to the flash descriptor [65].

**Anomaly Detection (DM-02):** This involves two key approaches: UEFI dynamic analysis to monitor for unusual activities [68], and UEFI static analysis [7, 114]. Dynamic analysis scrutinizes UEFI behavior during execution, monitoring run-time activities to detect suspicious or malicious actions. In contrast, static analysis examines UEFI firmware without execution, analyzing source code, binary code, and related files to identify potential vulnerabilities and insecure practices. Both methods use specialized tools and techniques for UEFI security analysis [39].

## 5.3 Mitigation Measures

Mitigation measures aim to address active threats and prevent further exploitation, playing a crucial role in reducing the impact of attacks.

**Incident Response (MM-01):** This involves investigating suspicious activities within the UEFI environment identified through preventive and detective controls [39]. This includes collecting and analyzing system logs, memory dumps, and other digital artifacts to identify the root cause of the incident, assess the extent of the compromise, and gather evidence for further analysis. This step also involves assessing the impact on the supply chain [8].

**Isolation (MM-02):** A key aspect of this countermeasure is isolating affected systems to prevent further infection. Additionally, isolating and disabling affected SMM features is important, as compromised SMM can lead to significant security risks [98]. Identify and disable compromised update paths to prevent unauthorized or malicious firmware updates. Isolation also involves assessing the integrity of UEFI update mechanisms, such as firmware update utilities, bootloaders, or firmware repositories, and disabling compromised update paths [12]. Similarly, analyzing security feature bypasses involves examining the effectiveness of UEFI security mechanisms and identifying potential ways attackers could bypass or exploit these features [39, 60].

**Credential Management (MM-03):** Effective credential management includes revoking compromised credentials and cryptographic keys used in the UEFI environment [25, 32, 40]. This involves disabling or revoking user accounts, access tokens, or digital certificates that have been compromised or suspected of compromise [13, 74].

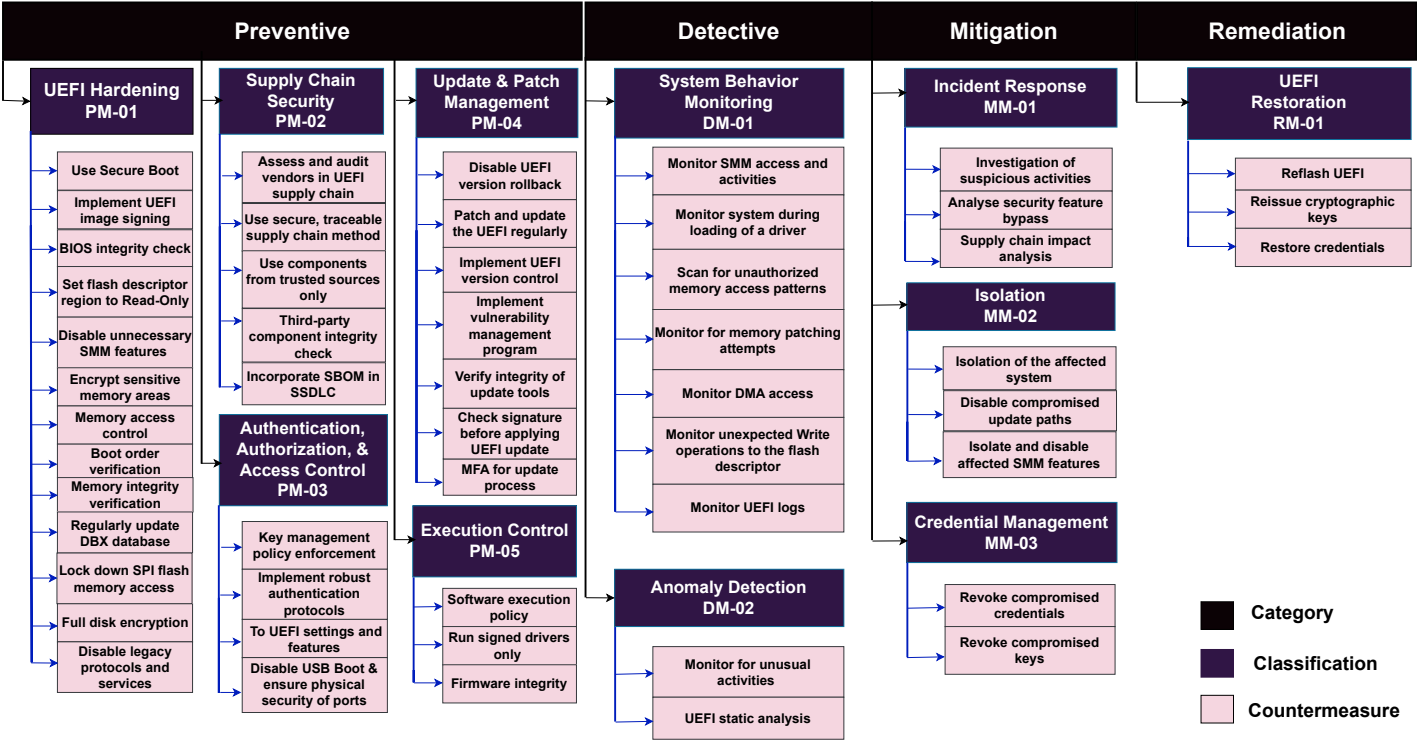


## 5.4 Remediation Measures

Effective remediation measures restore impacted systems and the integrity and security of the impacted UEFI environment.

**UEFI Restoration (RM-01):** UEFI restoration involves reflashing firmware to apply the latest updates and patches, addressing known vulnerabilities [12, 40, 44]. Re-issuing cryptographic keys for security mechanisms, such as secure boot or encryption, is recommended to invalidate compromised keys. Additionally, restoring credentials, including resetting passwords and implementing strong authentication mechanisms, is crucial to safeguarding UEFI system access.

Fig. 4. UEFI Countermeasures



6 Risk Analysis

We conducted a qualitative risk assessment considering multiple factors that influence the overall risk level [56]. Integrating impact, likelihood, and countermeasure effectiveness allows for a more comprehensive understanding of each technique’s potential consequences.

**Methodology:** We derived the risk level for each UEFI attack technique as a function of impact, likelihood, and effectiveness [77] based on the following matrices:

**Impact Analysis (I) [Very High, High, Moderate, Low, Very Low]:**

The impact of a threat is directly proportional to factors such as the nature of the threat, the value and sensitivity of the targeted assets, and the extent of the threat’s reach within the UEFI system. For example, in the context of UEFI, the impact of malicious payload injection into a DXE driver is far more severe than UEFI code analysis, as payload injection enables immediate and potentially far-reaching exploitation. In contrast, UEFI code analysis is a precursor to potential threats. Its impact is conditional on the subsequent use of discovered information for malicious purposes. We conducted an impact analysis for each technique based on the following criteria:

Table 4. Likelihood Assessment Matrix

| Score                                                                             | Level     | Description                                                                          |
|-----------------------------------------------------------------------------------|-----------|--------------------------------------------------------------------------------------|
|  | Very High | Full resources to execute the attack are at the attacker’s disposal                  |
|  | High      | The attack can succeed using the acquired resources with little effort               |
|  | Moderate  | Insufficient resources are available to complete the attack                          |
|  | Low       | Great effort is required to acquire the resources required to perform this technique |
|  | Very Low  | Highly unlikely to execute the technique given the limited time and resources        |

**Likelihood Assessment (L) [Very High, High, Moderate, Low, Very Low]:**

The likelihood of a risk materializing depends on the UEFI attack surface, the threat actor’s capability and intent, the presence of known vulnerabilities within the UEFI, and the effectiveness of UEFI countermeasures. We conducted a likelihood assessment for each of the possible techniques:

Table 5. Likelihood Assessment Matrix

| Score                                                                               | Level     | Description                                                                          |
|-------------------------------------------------------------------------------------|-----------|--------------------------------------------------------------------------------------|
|  | Very High | Full resources to execute the attack are at the attacker’s disposal                  |
|  | High      | The attack can succeed using the acquired resources with little effort               |
|  | Moderate  | Insufficient resources are available to complete the attack                          |
|  | Low       | Great effort is required to acquire the resources required to perform this technique |
|  | Very Low  | Highly unlikely to execute the technique given the limited time and resources        |

**Effectiveness Score (E) [Not Applicable, Low, Moderate, High]:**

To identify UEFI threats, we examined the attacks and vulnerabilities associated with UEFI (see Section 3). The study resulted in a common pattern observed in attacks and we mapped that to MITRE ATT&CK-like framework (refer to Figure 3). Next, we presented countermeasures for each UEFI attack technique (see Figure 4). For each countermeasure and attack technique pair, we assigned an applicability score (A 0–3) as:

Table 6. Applicability Score Matrix

| Score | Description                                                                                                                 |
|-------|-----------------------------------------------------------------------------------------------------------------------------|
| 0     | The countermeasure is not applicable                                                                                        |
| 1     | The countermeasure is useful but has an indirect impact                                                                     |
| 2     | The countermeasure is important and quite effective but may not cover all aspects to defend the given UEFI attack technique |
| 3     | The countermeasure is highly effective and plays a direct and critical role in ensuring UEFI security                       |

Next, for each UEFI attack technique  $t$ , we calculate the *effectiveness score*  $E(t)$  by aggregating the applicability scores assigned to the mapped countermeasures. Let  $a(t, c)$  denote the applicability score assigned to countermeasure  $c$  for attack technique  $t$ , according to Table 6, where  $a(t, c) \in \{0, 1, 2, 3\}$  and  $a(t, c) = 0$  indicates that the countermeasure is not applicable. Let  $C_t$  denote the set of applicable mapped countermeasures for technique  $t$ , and let  $N_t = |C_t|$  denote the number of such mapped countermeasures. The effectiveness score is computed as:

$$E(t) = \frac{1}{N_t} \sum_{c \in C_t} a(t, c) \tag{1}$$

If no applicable mapped countermeasure exists for a given technique, we set  $E(t) = 0$  by definition. This formulation ensures that the effectiveness score reflects the average defensive relevance of the countermeasures that are actually applicable to the given technique. Although the applicability scores themselves are discrete, the aggregated effectiveness score  $E(t)$  may take non-integer values because it is computed as an average. The applicability values are expert-assigned ordinal scores intended to approximate the relative defensive relevance of a countermeasure to a given UEFI attack technique, rather than empirically measured effect sizes. Accordingly, the resulting effectiveness values should be interpreted as structured comparative indicators for prioritization rather than as exact quantitative measurements.

We then map  $E(t)$  to the qualitative matrix scale as:

Table 7. Priority Levels Based on Effectiveness Score

| Effectiveness Tier | E Score Range |
|--------------------|---------------|
| Low Coverage       | 0 – 1.49      |
| Moderate Coverage  | 1.5 – 2.5     |
| Strong Coverage    | Above 2.5     |

The thresholds of 1.5 and 2.5 are grounded in the structure of the applicability scale. Since the scale ranges from 0 to 3, the value 1.5 marks the midpoint between low and moderate coverage, while 2.5 marks the upper region in which countermeasures on average approach strong, direct coverage. These thresholds therefore divide the 0–3 range into three semantically meaningful regions corresponding to low, moderate, and strong countermeasure coverage.

**Results of the Analysis:** The final qualitative risk level is derived from the joint consideration of impact ( $I$ ), likelihood ( $L$ ), and effectiveness  $E(t)$ . First, impact and likelihood are combined

to determine the baseline severity of a technique. In the revised analysis, we operationalize this baseline severity using the product  $I \times L$ : techniques with  $I \times L \geq 16$  are categorized as *High*, techniques with  $8 \leq I \times L < 16$  are categorized as *Moderate*, and techniques with  $I \times L < 8$  are categorized as *Low*. The effectiveness score then acts as an adjustment factor: techniques with weaker countermeasure coverage remain in higher risk tiers, whereas techniques with stronger countermeasure coverage may be placed in lower tiers despite having similar impact and likelihood. For interpretability, we group effectiveness into three broad coverage tiers using Table 7. We then derive the final risk level according to the rubric in Table 8.

Table 8. Rubric for Deriving the Final Risk Level

| Baseline Severity from $I$ and $L$ | Effectiveness Tier       | Final Risk |
|------------------------------------|--------------------------|------------|
| High                               | Low or Moderate Coverage | High       |
| High                               | Strong Coverage          | Mid        |
| Moderate                           | Low or Moderate Coverage | Mid        |
| Moderate                           | Strong Coverage          | Fair       |
| Low                                | Low or Moderate Coverage | Fair       |
| Low                                | Strong Coverage          | Low        |

The thresholds used for effectiveness and final risk grouping are intended to group techniques into broad comparative risk tiers for interpretability. They should therefore be understood as presentation-oriented cutoffs rather than exact quantitative boundaries. The risk analysis reported in this section is computed over the finalized set of techniques and sub-techniques included in the taxonomy presented in Section 4. The results of this analysis, presented in Table 9, indicate that several techniques remain highly exposed despite the presence of multiple relevant countermeasures. In particular, techniques such as malicious boot loader injection, malicious payload injection, malicious driver injection, physical attacks, and SMM exploitation remain in the highest qualitative risk tier. At the same time, a broader set of techniques fall into the mid-risk category, indicating that existing countermeasures provide partial but still incomplete coverage across much of the UEFI attack surface. This finding emphasizes the need for stronger layered defenses, more robust firmware-specific monitoring, and improved protection of the most security-critical stages of the UEFI lifecycle. While many techniques are meaningfully countered, no single control provides complete protection across the full UEFI threat landscape, reinforcing the need for a layered and adaptive security strategy.

Table 9. UEFI Risk Analysis Results

| # | UEFI Technique                                   | (I) | (L) | (E)  | Risk |
|---|--------------------------------------------------|-----|-----|------|------|
| 1 | Code Execution – Malicious Boot Loader Injection | 5   | 4   | 1.92 | High |
| 2 | Code Execution – Malicious Payload Injection     | 5   | 4   | 1.92 | High |

Continued on next page

| #  | UEFI Technique                                                   | (I) | (L) | (E)  | Risk     |
|----|------------------------------------------------------------------|-----|-----|------|----------|
| 3  | Physical Attacks                                                 | 5   | 4   | 2.00 | High     |
| 4  | Code Execution – Malicious Driver Injection                      | 5   | 4   | 2.08 | High     |
| 5  | SMM Exploitation                                                 | 5   | 4   | 2.17 | High     |
| 6  | Subvert UEFI Security Controls – UEFI Security Policy Alteration | 4   | 4   | 1.62 | High     |
| 7  | Bootkit Installation                                             | 4   | 4   | 1.79 | High     |
| 8  | Subvert UEFI Security Controls – Secure Boot Bypass              | 4   | 4   | 1.88 | High     |
| 9  | Exploit UEFI Services                                            | 5   | 3   | 1.50 | Mid      |
| 10 | Compromise Accounts                                              | 5   | 3   | 2.18 | Mid      |
| 11 | UEFI Image Tampering – Malicious Remote Update                   | 5   | 3   | 2.26 | Mid      |
| 12 | UEFI Image Tampering – Direct Flashing                           | 5   | 3   | 2.43 | Mid      |
| 13 | Hardware and UEFI Inventory                                      | 3   | 4   | 1.75 | Mid      |
| 14 | Network Attacks                                                  | 2   | 5   | 1.60 | Mid      |
| 15 | Supply Chain Identification – Distribution Chain                 | 3   | 3   | 2.20 | Mid      |
| 16 | Compromise Remote Update                                         | 5   | 2   | 2.28 | Mid      |
| 17 | UEFI Image Extraction                                            | 3   | 4   | 1.88 | Mid      |
| 18 | System Enumeration                                               | 3   | 4   | 1.62 | Mid      |
| 19 | Social Engineering                                               | 3   | 4   | 1.75 | Mid      |
| 25 | UEFI Services Exploitation                                       | 4   | 3   | 1.50 | Mid      |
| 27 | Develop Capabilities – Exploit Development                       | 3   | 3   | 1.22 | Mid      |
| 29 | Hooking UEFI API/Services                                        | 3   | 3   | 1.50 | Mid      |
| 30 | Hooking UEFI Services                                            | 3   | 3   | 1.50 | Mid      |
| 31 | System Architecture Mapping                                      | 3   | 3   | 1.50 | Mid      |
| 32 | UEFI Services Exploitation                                       | 4   | 3   | 1.50 | Mid      |
| 33 | Exploit Supply Chain – Hardware Supply Chain                     | 3   | 4   | 1.57 | Mid      |
| 34 | Leaked Authentic Digital Certificates                            | 4   | 2   | 1.54 | Mid      |
| 35 | Network Configuration and Credentials                            | 2   | 5   | 1.56 | Mid      |
| 36 | UEFI Code Analysis and Vulnerability Research                    | 3   | 4   | 1.60 | Mid      |
| 42 | Supply Chain Identification – Distribution Chain                 | 3   | 3   | 2.20 | Mid      |
| 43 | Supply Chain Identification – Third-Party Dependency             | 3   | 3   | 1.75 | Mid      |
| 44 | Overflow Vulnerability                                           | 4   | 3   | 1.75 | Mid      |
| 46 | DMA Attacks                                                      | 4   | 3   | 2.00 | Mid      |
| 20 | Boot Configuration Analysis                                      | 3   | 3   | 1.17 | Moderate |

Continued on next page

| #  | UEFI Technique                                         | (I) | (L) | (E)  | Risk     |
|----|--------------------------------------------------------|-----|-----|------|----------|
| 21 | Hooking UEFI Services                                  | 3   | 3   | 1.50 | Moderate |
| 22 | System Architecture Mapping                            | 3   | 3   | 1.50 | Moderate |
| 23 | Peripheral Device Discovery                            | 3   | 4   | 1.29 | Moderate |
| 26 | Abusing Secure Boot                                    | 3   | 3   | 1.27 | Moderate |
| 37 | Code Signing Certificate Abuse                         | 4   | 2   | 1.67 | Moderate |
| 38 | UEFI Code Analysis and Vulnerability Research          | 3   | 4   | 1.60 | Moderate |
| 39 | Third-Party Dependency                                 | 3   | 3   | 1.75 | Moderate |
| 40 | Acquire Access                                         | 4   | 2   | 1.82 | Moderate |
| 45 | UEFI Shell Exploitation                                | 3   | 3   | 1.57 | Moderate |
| 24 | Manipulating UEFI Variables                            | 3   | 2   | 1.40 | Fair     |
| 41 | Supply Chain Identification – UEFI Development Process | 3   | 2   | 1.75 | Fair     |
| 47 | Platform Key Abuse                                     | 3   | 1   | 1.60 | Fair     |
| 48 | UEFI Variable Tampering                                | 3   | 2   | 1.40 | Fair     |
| 49 | Code Obfuscation                                       | 2   | 4   | 1.25 | Fair     |
| 50 | Payload Removal                                        | 2   | 4   | 1.17 | Fair     |
| 51 | Digital Certificate and Key Extraction                 | 3   | 2   | 1.40 | Fair     |
| 52 | UEFI Variable Enumeration                              | 2   | 2   | 1.00 | Fair     |

## 7 Limitations

This work makes several contributions to the understanding of UEFI security; however, several limitations should be acknowledged to contextualize the findings appropriately.

**Subjectivity of Risk Scores.** The impact (I) and likelihood (L) scores assigned to each attack technique were derived through structured expert judgment informed by the reviewed attack corpus, rather than through a formal elicitation process such as a Delphi study. While this approach is consistent with established applied risk assessment frameworks including NIST SP 800-30 [77], it introduces subjectivity that could affect individual technique rankings. Specifically, techniques for which limited empirical data is available may have scores that reflect the assessed severity of the threat more than its observed frequency. Future work should validate and refine these scores through structured expert surveys involving practitioners from across the UEFI ecosystem, including IBVs, OEMs, and security researchers, to reduce reliance on individual judgment and improve inter-rater reliability.

**Bias Toward Publicly Documented Attacks.** The attack corpus underlying the taxonomy and risk analysis is necessarily limited to attacks that have been publicly disclosed. State-sponsored attacks, undisclosed zero-day exploits, and techniques employed by advanced persistent threat actors that have not entered the public domain are not represented. This introduces a systematic bias toward attacks that were either detected, disclosed by vendors, or deliberately made public by threat actors. As a consequence, the taxonomy may underrepresent sophisticated, covert techniques that have not yet been observed or reported. The risk scores of techniques associated with such covert activity, such as hardware supply chain compromise (UEFI-T0009.002), may therefore be

underestimated relative to their true prevalence. We acknowledge this limitation and note that it is inherent to any empirical security taxonomy grounded in observed attacks rather than red-team simulation.

**Countermeasures Analysis Does Not Evaluate Actual Deployment.** The countermeasures presented in Section 5 and the applicability scores used in the effectiveness calculation reflect the theoretical coverage of each control against each attack technique. They do not account for the degree to which these countermeasures are actually deployed in practice, nor for the configuration quality of those that are deployed. For example, Secure Boot is widely supported but frequently misconfigured or disabled in real-world deployments, meaning its practical effectiveness is lower than its theoretical applicability score would suggest. A deployment-aware analysis would require empirical data on real-world UEFI configurations at scale, which is not currently available in the public domain. Future research should seek to close this gap through large-scale firmware analysis studies.

**Taxonomy Validation.** The proposed taxonomy was derived systematically from the reviewed attack corpus and iteratively refined against the full set of documented attacks, as described in SubSection 4.1. However, it has not undergone formal external validation by domain experts independent of the authorship team. Expert validation, such as through a structured Delphi process or red-team evaluation, would strengthen confidence in the taxonomy’s completeness, the appropriateness of technique groupings, and the accuracy of tactic boundaries. We present the taxonomy as a foundational framework intended to evolve with community input, and explicitly invite validation and extension by the UEFI security research community. Formal validation is identified as a priority direction for future work in Subsection 8.1.

## 8 Conclusion and Future Research

The landscape of UEFI security has evolved dramatically over the past decade, with attacks becoming increasingly sophisticated and pervasive. Our analysis demonstrates that threat actors targeting UEFI firmware have progressed from well-resourced state actors to a broader range of malicious entities, including cybercriminal groups. This evolution is reflected by the increasing complexity of attacks, such as BlackLotus and MoonBounce, which are challenging to detect and mitigate.

However, our risk analysis reveals a gap in the current security landscape. While several preventive measures exist—such as Secure Boot and firmware integrity checks—there is a notable lack of dynamic security mechanisms capable of adapting to the evolving threat landscape. Static defenses, while necessary, are increasingly insufficient against modern UEFI attacks. This disparity between attack sophistication and defense capabilities underscores the need for adaptive, real-time security solutions. Future research should focus on creating dynamic defense mechanisms that can detect and respond to UEFI-related attack patterns in real time, as outlined below.

### 8.1 Future Research Directions

Our analysis reveals that the primary gaps in UEFI security are not in the absence of controls, but in their lack of adaptability, validation, and deployment assurance. Building on the taxonomy and risk analysis presented in this work, we identify several key research directions that warrant further investigation.

**Dynamic and Adaptive Defense Mechanisms.** Our risk analysis shows that high-risk techniques such as malicious payload injection, SMM exploitation, and malicious driver injection remain insufficiently mitigated despite the presence of preventive controls. This gap stems from an over-reliance on static, signature-based defenses. Future work should focus on runtime, firmware-level monitoring mechanisms capable of detecting deviations in boot flow execution, unauthorized modifications to UEFI services, and anomalous SPI flash access patterns. Such systems should



operate across early boot phases, including PEI and DXE, and respond autonomously without relying on OS-level visibility.

**Automated Vulnerability Discovery in UEFI Firmware.** Reconnaissance techniques such as UEFI code analysis (UEFI-T0002) and hardware inventory (UEFI-T0003) are actively leveraged by adversaries. However, current analysis of UEFI firmware remains largely manual or limited to small-scale tools. Future research should develop scalable static and dynamic analysis pipelines tailored to firmware images, building on prior efforts such as symbolic execution and tools like efiXplorer. Particular attention is needed for third-party DXE drivers and Option ROMs, which represent a significant and under-examined attack surface.

**Supply Chain Security and Verification.** As discussed throughout the paper, the UEFI ecosystem depends on a multi-actor supply chain spanning chipset manufacturers, IBVs, ODMs, and OEMs. This distributed development and deployment model introduces multiple points of potential compromise across firmware creation, integration, update, and distribution. Future work should investigate formal verification techniques, reproducible firmware build pipelines, and cryptographic attestation mechanisms to strengthen integrity guarantees across the supply chain. Additionally, practical frameworks for integrating Software Bill of Materials (SBOM) into UEFI firmware releases remain an open challenge.

**System Management Mode (SMM) Security.** SMM exploitation (UEFI-T0033) ranks among the highest-risk techniques in our analysis, yet existing defenses remain limited, largely proprietary, and difficult to audit. Future research should explore formal isolation guarantees for SMM handlers, systematic fuzzing of SMM communication interfaces, and hardware-assisted enforcement of memory access policies to reduce the attack surface of this highly privileged execution environment.

**Emerging Technologies as Defensive Enablers.** Several nascent technologies offer promising avenues for strengthening UEFI security and warrant dedicated investigation. Hardware-based roots of trust, such as Microsoft Pluton and Google Titan, provide cryptographic attestation capabilities that are physically isolated from the SPI flash chip and may offer stronger integrity guarantees than software-only approaches. Confidential computing frameworks, including AMD SEV and Intel TDX, extend hardware-assisted isolation to firmware execution environments, potentially limiting the blast radius of SMM exploitation (UEFI-T0033) and malicious driver injection (UEFI-T0016.001). Post-quantum cryptography represents another critical frontier: as current Secure Boot implementations rely on RSA and elliptic curve schemes, future-proofing firmware signing against quantum-capable adversaries requires timely migration to quantum-resistant algorithms. Finally, machine learning-based anomaly detection applied to boot-time telemetry, such as deviations in measured boot logs or unexpected patterns in UEFI variable access, offers a path toward the adaptive, real-time defenses identified as a priority throughout this paper.

**Standardized UEFI Security Testing and Certification.** Currently, no publicly available benchmark suite exists for evaluating UEFI firmware security against known attack patterns. Developing a standardized testing and certification framework grounded in the taxonomy proposed in this paper would enable vendors to systematically assess firmware robustness. Such efforts could be analogous to existing validation programs in other domains and would provide enterprises with a consistent basis for comparing platform security.

**Regulations, Standards, and Policy Enforcement.** Because the UEFI ecosystem spans multiple stakeholders, including chipset manufacturers, IBVs, ODMs, OEMs, and enterprise operators, technical controls alone are insufficient to ensure consistent security outcomes. Future research should therefore also examine the role of regulatory requirements, certification schemes, and organizational policy frameworks in improving UEFI security. Examples include mandating secure firmware development practices, requiring supply-chain audits, enforcing coordinated vulnerability disclosure and patch management processes, and establishing minimum integrity and provenance

requirements for firmware updates. Such measures can complement technical defenses by improving accountability, transparency, and baseline security assurance across the firmware lifecycle.

**Taxonomy Validation and Extension.** The MITRE ATT&CK-like taxonomy introduced in this paper is derived from real-world attacks and proof-of-concept implementations, but has not yet undergone formal validation. Future work should involve expert-driven validation, such as Delphi studies, and red-team evaluations to refine and extend the taxonomy. In particular, emerging attack vectors targeting ARM-based platforms, virtualized firmware environments, and cloud-hosted systems should be incorporated to ensure long-term relevance.

**Operational Security and End-User Awareness.** Finally, many UEFI security mechanisms, including Secure Boot, are often misconfigured or disabled in practice. Future research should explore usable security approaches and deployment guidelines that ensure correct configuration, maintenance, and adoption of existing protections in real-world environments.

## References

- [1] Security Affairs. 2020. TRICKBOOT FEATURE ALLOWS TRICKBOT BOT TO RUN UEFI ATTACKS. <https://securityaffairs.com/111849/malware/trickbot-trickboot-uefi-attacks.html>
- [2] Alex Matrosov. 2017. UEFI Firmware Rootkits Myths and Reality. <https://www.blackhat.com/docs/asia-17/materials/asia-17-Matrosov-The-UEFI-Firmware-Rootkits-Myths-And-Reality.pdf>
- [3] Kai Michaelis Alex Matrosov, Richard Hughes. 2023. The Various Shades of Supply Chain: SBOM, N-Days and Zero Trust. Available online at: <https://www.youtube.com/watch?v=DwCgPjvssD4>. <https://www.blackhat.com/asia-23/briefings/schedule/#the-various-shades-of-supply-chain-sbom-n-days-and-zero-trust-31253>.
- [4] Palo Alto. n.d.. Diving Into Glupteba's UEFI Bootkit. <https://unit42.paloaltonetworks.com/glupteba-malware-uefi-bootkit/>.
- [5] Assaf Carlsbad. 2020. Moving From Common-Sense Knowledge About UEFI To Actually Dumping UEFI Firmware. <https://www.sentinelone.com/labs/moving-from-common-sense-knowledge-about-uefi-to-actually-dumping-uefi-firmware/>. Accessed: 2024-08-01.
- [6] Vladimir Bashun, Anton Sergeev, Victor Minchenkov, and Alexandr Yakovlev. 2013. Too young to be secure: Analysis of UEFI threats and vulnerabilities. 16–24. <https://doi.org/10.1109/FRUCT.2013.6737940>
- [7] Binarly. n.d.. Using Symbolic Execution to Detect UEFI Firmware Vulnerabilities. <https://www.binarly.io/blog/using-symbolic-execution-to-detect-uefi-firmware-vulnerabilities>.
- [8] Blackhat. 2021. Safeguarding UEFI Ecosystem Firmware Supply Chain is Hardcoded. <https://i.blackhat.com/USA21/Wednesday-Handouts/us-21-Safeguarding-UEFI-Ecosystem-Firmware-Supply-Chain-Is-Hardcoded.pdf>.
- [9] Yakup Cengiz. 2020. The Evolution from BIOS to UEFI. <https://medium.com/@yakupcengiz/the-evolution-from-bios-to-uefi-cd5576f8279e> <https://medium.com/@yakupcengiz/the-evolution-from-bios-to-uefi-cd5576f8279e>.
- [10] Cr4sh. 2016. Lenovo ThinkPad System Management Mode arbitrary code execution exploit. <https://github.com/Cr4sh/ThinkPwn>
- [11] Cr4sh. 2022. PEI stage backdoor for UEFI compatible firmware. <https://github.com/Cr4sh/PeiBackdoor>
- [12] Cybersecurity and Infrastructure Security Agency (CISA). 2024. Federal Government Cybersecurity Incident and Vulnerability Response Playbooks. [https://www.cisa.gov/sites/default/files/2024-03/Federal\\_Government\\_Cybersecurity\\_Incident\\_and\\_Vulnerability\\_Response\\_Playbooks\\_508C.pdf](https://www.cisa.gov/sites/default/files/2024-03/Federal_Government_Cybersecurity_Incident_and_Vulnerability_Response_Playbooks_508C.pdf).
- [13] Cybersecurity and Infrastructure Security Agency (CISA). n.d.. Call to Action: Bolster UEFI Cybersecurity Now. <https://www.cisa.gov/news-events/news/call-action-bolster-uefi-cybersecurity-now>.
- [14] Eclipsium. 2019. SHADOWHAMMER AND THE FIRMWARE SUPPLY CHAIN. <https://eclipsium.com/wp-content/uploads/ShadowHammer-and-the-Firmware-Supply-Chain.pdf>
- [15] Eclipsium. [n. d.]. Firmware Security Realizations - Part 1: Secure Boot and DBX. <https://eclipsium.com/research/firmware-security-realizations-part-1-secure-boot-and-dbx/>. Accessed: 2024-08-01.
- [16] Eclipsium. 2020. PROTECT YOUR ORGANIZATION FROM MOSAICREGRESSOR AND OTHER UEFI IMPLANTS. <https://eclipsium.com/wp-content/uploads/Protecting-Your-Organizations-From-MosaicRegressor.pdf>
- [17] Eclipsium. 2020. THERE'S A HOLE IN THE BOOT. <https://eclipsium.com/blog/theres-a-hole-in-the-boot/>
- [18] Eclipsium. 2020. TRICKBOT NOW OFFERS 'TRICKBOOT': PERSIST, BRICK, PROFIT. <https://eclipsium.com/wp-content/uploads/TrickBot-Now-Offers-TrickBoot-Persist-Brick-Profit.pdf>
- [19] Eclipsium. 2021. FINSPY UEFI AND MBR BOOTKIT. <https://eclipsium.com/blog/finspy-uefi-and-mbr-bootkit/>
- [20] Eclipsium. 2022. YET ANOTHER UEFI BOOTKIT DISCOVERED: MEET COSMICSTRAND. <https://eclipsium.com/blog/yet-another-uefi-bootkit-discovered-meet-cosmicstrand/>

- [21] Eclypsium. 2024. What You Need to Know About the Latest KEV Updates. <https://eclypsium.com/blog/what-you-need-to-know-about-the-latest-kev-updates/>. Accessed: 2024-08-08.
- [22] Eclypsium. n.d. Direct Memory Access Attacks: A Walk Down Memory Lane. <https://eclypsium.com/research/direct-memory-access-attacks-a-walk-down-memory-lane/>.
- [23] Eclypsium. n.d. Evil Maid Firmware Attacks Using USB Debug. <https://eclypsium.com/research/evil-maid-firmware-attacks-using-usb-debug/>.
- [24] Eclypsium. n.d. Firmware Security Realizations Part 3: SPI Write Protections. <https://eclypsium.com/research/firmware-security-realizations-part-3-spi-write-protections/>.
- [25] Eclypsium. n.d. Sunburst Firmware Supply Chain Attack IR. <https://eclypsium.com/blog/sunburst-firmware-supply-chain-attack-ir/>.
- [26] Eduard Kovacs. 2017. WikiLeaks Releases Data on CIA's Apple Hacking Tools. <https://www.securityweek.com/wikileaks-releases-data-cias-apple-hacking-tools/>
- [27] Entitle. n.d.. Multi-Factor Authentication (MFA). <https://www.entitle.io/resources/glossary/multi-factor-authentication-mfa>.
- [28] ESET. 2021. ESET Research discovers ESpecter, a UEFI bootkit for cyberespionage. <https://www.eset.com/in/about/newsroom/press-releases/research/eset-research-discovers-especter-a-uefi-bootkit-for-cyberespionage/>
- [29] CERT EU. 2019. UPDATE: Operation ShadowHammer – Compromised ASUS Computers. <https://cert.europa.eu/publications/security-advisories/2019-007/>
- [30] Firmware Security. 2017. Wikileaks: Vault 7: Dark Matter. <https://firmwaresecurity.com/tag/darkmatter/>
- [31] UEFI Forum. 2012. UEFI Plugfest 2012Q1 v3 AMI. [https://uefi.org/sites/default/files/resources/UEFI\\_Plugfest\\_2012Q1\\_v3\\_AMI.pdf](https://uefi.org/sites/default/files/resources/UEFI_Plugfest_2012Q1_v3_AMI.pdf).
- [32] UEFI Forum. 2013. UEFI Secure Boot in Modern Computer Security Solutions. [https://uefi.org/sites/default/files/resources/UEFI\\_Secure\\_Boot\\_in\\_Modern\\_Computer\\_Security\\_Solutions\\_2013.pdf](https://uefi.org/sites/default/files/resources/UEFI_Secure_Boot_in_Modern_Computer_Security_Solutions_2013.pdf).
- [33] UEFI Forum. 2014. UEFI Plugfest 2014 Phoenix. [https://uefi.org/sites/default/files/resources/2014\\_UEFI\\_Plugfest\\_06\\_Phoenix.pdf](https://uefi.org/sites/default/files/resources/2014_UEFI_Plugfest_06_Phoenix.pdf).
- [34] UEFI Forum. 2022. UEFI Virtual Summit 2022: Tackling Security Through the Supply Chain. [https://uefi.org/sites/default/files/resources/UEFI\\_VirtualSummit\\_2022\\_Tackling\\_Security\\_Through\\_the\\_Supply\\_Chain\\_Draft\\_Final\\_v4.pdf](https://uefi.org/sites/default/files/resources/UEFI_VirtualSummit_2022_Tackling_Security_Through_the_Supply_Chain_Draft_Final_v4.pdf).
- [35] UEFI Forum. n.d.. Hardening the Core Enhanced Memory Protection. [https://uefi.org/sites/default/files/resources/Hardening%20the%20Core%20Enhanced%20Memory%20Protection\\_Beebe.pdf](https://uefi.org/sites/default/files/resources/Hardening%20the%20Core%20Enhanced%20Memory%20Protection_Beebe.pdf)
- [36] UEFI Forum. n.d.. Implementing and Using the UEFI KMS. [https://uefi.org/sites/default/files/resources/Implementing%20and%20Using%20the%20UEFI%20KMS\\_AMI\\_Final.pdf](https://uefi.org/sites/default/files/resources/Implementing%20and%20Using%20the%20UEFI%20KMS_AMI_Final.pdf)
- [37] UEFI Forum. n.d.. Intel Attacking and Defending the Platform. [https://uefi.org/sites/default/files/resources/Intel\\_Attacking%20and%20Defending%20the%20Platform.pdf](https://uefi.org/sites/default/files/resources/Intel_Attacking%20and%20Defending%20the%20Platform.pdf)
- [38] UEFI Forum. n.d.. Secure Boot and Driver Signing. [https://uefi.org/specs/UEFI/2.9\\_A/32\\_Secure\\_Boot\\_and\\_Driver\\_Signing.html](https://uefi.org/specs/UEFI/2.9_A/32_Secure_Boot_and_Driver_Signing.html).
- [39] UEFI Forum. n.d.. UEFI Firmware - Security Concerns and Best Practices. <https://uefi.org/sites/default/files/resources/UEFI%20Firmware%20-%20Security%20Concerns%20and%20Best%20Practices.pdf>
- [40] UEFI Forum. n.d.. UEFI Key Management Service (KMS) With TPM. [https://uefi.org/sites/default/files/resources/UEFI%20Key%20Management%20Service%20%28KMS%29%20With%20TPM%C2%A0\\_Otumfuor%20and%20Polyudov.pdf](https://uefi.org/sites/default/files/resources/UEFI%20Key%20Management%20Service%20%28KMS%29%20With%20TPM%C2%A0_Otumfuor%20and%20Polyudov.pdf)
- [41] UEFI Forum. n.d.. UEFI March Webinar Secure Coding for UEFI Firmware. <https://uefi.org/sites/default/files/resources/UEFI%20March%20Webinar%20Secure%20Coding%20for%20UEFI%20Firmware.pdf>
- [42] UEFI Forum. n.d.. UEFI Support for Software Bill of Materials (SBOM). [https://uefi.org/sites/default/files/resources/UEFI%20Support%20for%20Software%20Bill%20of%20Materials%20%28SBOM%29\\_AMI\\_UEFI\\_Final\\_Deck.pdf](https://uefi.org/sites/default/files/resources/UEFI%20Support%20for%20Software%20Bill%20of%20Materials%20%28SBOM%29_AMI_UEFI_Final_Deck.pdf)
- [43] UEFI Forum. n.d.. UPFS11 P2 SecureBoot Insyde. [https://uefi.org/sites/default/files/resources/UPFS11\\_P2\\_SecureBoot\\_Insyde.pdf](https://uefi.org/sites/default/files/resources/UPFS11_P2_SecureBoot_Insyde.pdf).
- [44] UEFI Forum. n.d.. Vulnerability Management in UEFI. [https://uefi.org/sites/default/files/resources/Vulnerability%20Management%20in%20UEFI\\_Mullen.pdf](https://uefi.org/sites/default/files/resources/Vulnerability%20Management%20in%20UEFI_Mullen.pdf)
- [45] Julia Glazova. 2022. CosmicStrand: a UEFI rootkit. <https://www.kaspersky.com/blog/cosmicstrand-uefi-rootkit/45017/>
- [46] Cloud Security Industry Summit Supply Chain Technical Working Group. [n. d.]. Secure Firmware Development Best Practices. <https://www.opencompute.org/documents/csis-firmware-security-best-practices-position-paper-version-1-0-pdf>. Accessed: 2024-08-01.
- [47] hackedteam. 2015. EFI Development Kit. <https://github.com/hackedteam/vector-edk>
- [48] Jakob Hagl, Oliver Mann, and Martin Pirker. 2021. Securing the Linux Boot Process: From Start to Finish. In *Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP)*. 604–610.

- [49] hp. 2021. ROOTKIT TECHNICAL WHITEPAPER AN ANALYSIS OF THE MALWARE LOJAX UEFI OVERVIEW. <https://h20195.www2.hp.com/v2/GetDocument.aspx?docname=4AA7-4019ENW>
- [50] Trammell Hudson and Larry Rudolph. 2015. Thunderstrike: EFI firmware bootkits for Apple MacBooks. In *Proceedings of the 8th ACM International Systems and Storage Conference*. 1–10.
- [51] Hudson, Trammell . 2014. Thunderstrike. <https://trmm.net/Thunderstrike/>
- [52] Hudson, Trammell . 2015. Thunderstrike. [https://trmm.net/Thunderstrike2\\_details/](https://trmm.net/Thunderstrike2_details/)
- [53] Ionut Ilascu. 2020. TrickBot's new 'TrickBoot' module infects your UEFI firmware. <https://www.bleepingcomputer.com/news/security/trickbots-new-trickboot-module-infects-your-uefi-firmware/>
- [54] Intel. [n. d.]. UEFI Secure Boot in Linux. <https://www.intel.com/content/dam/develop/external/us/en/documents/sf13-stts002-100p-820238.pdf>. Accessed: 2024-08-01.
- [55] Intel. 2011. UEFI Security and Networking Advancements. <https://www.intel.com/content/dam/develop/external/us/en/documents/sf11-efis001-100-820238.pdf>
- [56] ISACA. 2021. Risk Assessment and Analysis Methods: Qualitative and Quantitative. <https://www.isaca.org/resources/isaca-journal/issues/2021/volume-2/risk-assessment-and-analysis-methods>. Accessed: 2024-08-01.
- [57] Weihua Jiao, Qingbao Li, Zhifeng Chen, and Fei Cao. 2022. UEFI Security Threats Introduced by S3 and Mitigation Measure. In *2022 7th International Conference on Signal and Image Processing (ICSIP)*. IEEE, 734–740.
- [58] Sandra Radesky Jonathan Spring. 2023. A Call to Action: Bolster UEFI Cybersecurity Now. <https://www.cisa.gov/news-events/news/call-action-bolster-uefi-cybersecurity-now>
- [59] Sabastien Kaczmarek. 2013. UEFI and Dreamboot. In *Hack in the Box Security Conference, Kuala Lumpur, Malaysia*.
- [60] Corey Kallenberg, Sam Cornwell, Xeno Kovah, and John Butterworth. 2014. Setup for failure: defeating secure boot. In *The Symposium on Security for Asia Network (SyScan)(April 2014)*.
- [61] Corey Kallenberg, Xeno Kovah, John Butterworth, and Sam Cornwell. 2014. Extreme privilege escalation on Windows 8/UEFI systems. *BlackHat, Las Vegas, USA (2014)*.
- [62] Kaspersky. 2019. Operation ShadowHammer: new supply chain attack threatens hundreds of thousands of users worldwide. [https://www.kaspersky.com/about/press-releases/2019\\_operation-shadowhammer-new-supply-chain-attack](https://www.kaspersky.com/about/press-releases/2019_operation-shadowhammer-new-supply-chain-attack)
- [63] Kaspersky. 2020. Malware delivery through UEFI bootkit with MosaicRegressor. <https://www.kaspersky.com/blog/mosaicregressor-uefi-malware/37252/>
- [64] Kaspersky. 2021. FinFisher spyware improves its arsenal with four levels of obfuscation, UEFI infection and more. [https://www.kaspersky.com/about/press-releases/2021\\_finfisher-spyware-improves-its-arsenal-with-four-levels-of-obfuscation-uefi-infection-and-more](https://www.kaspersky.com/about/press-releases/2021_finfisher-spyware-improves-its-arsenal-with-four-levels-of-obfuscation-uefi-infection-and-more)
- [65] Xeno Kovah and Corey Kallenberg. [n. d.]. Advanced x86: BIOS and System Management Mode Internals Flash Descriptor. [https://opensecuritytraining.info/IntroBIOS\\_files/Day2\\_02\\_Advanced%20x86%20-%20BIOS%20and%20SMM%20Internals%20-%20Flash%20Descriptor.pdf](https://opensecuritytraining.info/IntroBIOS_files/Day2_02_Advanced%20x86%20-%20BIOS%20and%20SMM%20Internals%20-%20Flash%20Descriptor.pdf). Accessed: 2024-08-01.
- [66] Xeno Kovah and Corey Kallenberg. 2015. Are you giving firmware attackers a free pass. In *Proceedings of the RSA Conference, San Francisco, CA, USA*. 20–24.
- [67] Mikhail Krichanov and Vitaly Cheptsov. 2021. UEFI virtual machine firmware hardening through snapshots and attack surface reduction. In *2021 Ivannikov Ispras Open Conference (ISPRAS)*. IEEE, 30–36.
- [68] Andrei Labunets. 2020. efiXplorer: Hunting For UEFI Firmware Vulnerabilities At Scale With Automated Static Analysis. <https://i.blackhat.com/eu-20/Wednesday/eu-20-Labunets-efiXplorer-Hunting-For-UEFI-Firmware-Vulnerabilities-At-Scale-With-Automated-Static-Analysis.pdf>.
- [69] Ravie Lakshmanan. 2020. TrickBot Malware Gets UEFI/BIOS Bootkit Feature to Remain Undetected. <https://thehackernews.com/2020/12/trickbot-malware-gets-uefibios-bootkit.html>
- [70] MARK LECHTIK. 2022. MoonBounce: the dark side of UEFI firmware. <https://securelist.com/moonbounce-the-dark-side-of-uefi-firmware/105468/>
- [71] Secure List. 2022. CosmicStrand: the discovery of a sophisticated UEFI firmware rootkit. <https://securelist.com/cosmicstrand-uefi-firmware-rootkit/106973/>
- [72] Alex Matrosov, Eugene Rodionov, and Sergey Bratus. 2019. *Rootkits and bootkits: reversing modern malware and next generation threats*. No Starch Press.
- [73] Trend Micro. 2018. LoJax UEFI Rootkit Used in Cyberespionage. <https://www.trendmicro.com/vinfo/us/security/news/cyber-attacks/lojax-uefi-rootkit-used-in-cyberespionage>
- [74] Microsoft. 2023. KB5025885: How to Manage the Windows Boot Manager Revocations for Secure Boot Changes Associated with CVE-2023-24932. <https://support.microsoft.com/en-us/topic/kb5025885-how-to-manage-the-windows-boot-manager-revocations-for-secure-boot-changes-associated-with-cve-2023-24932-41a975df-beb2-40c1-99a3-b3ff139f832d>.
- [75] MITRE. 2023. MITRE ATT&CK. <https://attack.mitre.org/>

- [76] National Institute of Standards and Technology. 2020. *Cybersecurity White Paper: Securing Property Using Property-Based Attestation*. Technical Report. National Institute of Standards and Technology, Gaithersburg, MD. <https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.29.pdf> Accessed: August 21, 2024.
- [77] NIST. [n.d.]. NIST Special Publication 800-30 Revision 1. <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf> <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf>.
- [78] National Security Agency (NSA). n.d.. UEFI Defensive Practices Guidance. <https://www.nsa.gov/portals/75/documents/what-we-do/cybersecurity/professional-resources/ctr-uefi-defensive-practices-guidance.pdf>.
- [79] Magnus Nyström, Martin Nicholes, and Vincent J Zimmer. 2011. UEFI Networking and Pre-OS Security. *Intel Technology Journal* 15, 1 (2011).
- [80] Department of Defense. 2019. Enforce Signed Software Execution Policies. <https://media.defense.gov/2019/Sep/09/2002180334/-1/-1/0/Enforce%20Signed%20Software%20Execution%20Policies%20-%20Copy.pdf>
- [81] Department of Defense. 2023. CTR-UEFI Secure Boot Customization. <https://media.defense.gov/2023/Mar/20/2003182401/-1/-1/0/CTR-UEFI-SECURE-BOOT-CUSTOMIZATION-20230317.PDFnote>.
- [82] National Institute of Standards and Technology (NIST). 2011. NIST Special Publication 800-147: BIOS Protection Guidelines. <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-147.pdf>.
- [83] Charlie Osborne. 2021. UEFI threats moving to the ESP: Introducing ESpecter bootkit. <https://www.welivesecurity.com/2021/10/05/uefi-threats-moving-esp-introducing-especter-bootkit/>
- [84] OWASP. [n.d.]. Broken Access Control. [https://owasp.org/Top10/A01\\_2021-Broken\\_Access\\_Control/](https://owasp.org/Top10/A01_2021-Broken_Access_Control/). Accessed: 2024-08-01.
- [85] Vinay Pamnani. 2023. BitLocker Countermeasures. <https://learn.microsoft.com/en-us/windows/security/operating-system-security/data-protection/bitlocker/bitlocker-countermeasures#uefi-and-secure->
- [86] quarkslab. 2017. UEFI bootkit. <https://github.com/quarkslab/dreamboot>
- [87] Rapid7. n.d.. Vulnerability Management Program Framework. <https://www.rapid7.com/fundamentals/vulnerability-management-program-framework/>.
- [88] ESET Research. 2018. LoJax: First UEFI rootkit found in the wild, courtesy of the Sednit group. <https://www.welivesecurity.com/2018/09/27/lojax-first-uefi-rootkit-found-wild-courtesy-sednit-group/>
- [89] DH Eugene Rodionov, Aleksandr Matrosov, and David Harley. 2014. Bootkits: Past, present and future. In *VB Conference*.
- [90] Vijay Sarvepalli. 2023. Securing UEFI: An Underpinning Technology for Computing. (2023).
- [91] Tara Seals. 2022. Rare CosmicStrand UEFI Rootkit Swings into Cybercrime Orbit. <https://www.darkreading.com/endpoint-security/rare-cosmicstrand-uefi-rootkit-cybercrime-orbit>
- [92] securelist. 2019. Operation ShadowHammer: a high-profile supply chain attack. <https://securelist.com/operation-shadowhammer-a-high-profile-supply-chain-attack/90380/>
- [93] Securelist. 2020. MosaicRegressor: Lurking in the Shadows of UEFI. <https://securelist.com/mosaicregressor/98849/>
- [94] SecureList. 2021. FinSpy: unseen findings. <https://securelist.com/finspy-unseen-findings/104322/>
- [95] Martin Smolár. 2021. Meet ESpecter: a new UEFI bootkit for cyber spying. <https://www.zdnet.com/article/meet-especter-a-new-uefi-bootkit-for-cyber-spying/>
- [96] Martin Smolár. 2022. BlackLotus UEFI bootkit: Myth confirmed. <https://www.welivesecurity.com/2023/03/01/blacklotus-uefi-bootkit-myth-confirmed/>
- [97] Spiceworks. n.d.. What is Version Control? <https://www.spiceworks.com/tech/devops/articles/what-is-version-control/>.
- [98] Krzysztof Szczypiorski and Michał Szaknis. 2022. The design of the simple SMM rootkit. In *9th International Conference on Wireless Communication and Sensor Networks*. Association for Computing Machinery.
- [99] Takahiro Haruyama. 2021. Detecting UEFI Bootkits in the Wild (Part 1). <https://blogs.vmware.com/security/2021/06/detecting-uefi-bootkits-in-the-wild-part-1.html>
- [100] TCG. 2016. UTCG EFI Protocol Specification. <https://trustedcomputinggroup.org/wp-content/uploads/EFI-Protocol-Specification-rev13-160330final.pdf>.
- [101] Robert Bruce Thompson and Barbara Fritchman Thompson. 2003. *PC hardware in a nutshell: a desktop quick reference*. "O'Reilly Media, Inc".
- [102] Bill Toulas. 2022. New MoonBounce UEFI malware used by APT41 in targeted attacks. <https://www.bleepingcomputer.com/news/security/new-moonbounce-uefi-malware-used-by-apt41-in-targeted-attacks/>
- [103] UEFI. 2010. UEFI MFA. [https://uefi.org/specs/UEFI/2.9\\_A/36\\_User\\_Identification.html](https://uefi.org/specs/UEFI/2.9_A/36_User_Identification.html)
- [104] UEFI. 2016. UEFI Shell Specification. [https://uefi.org/sites/default/files/resources/UEFI\\_Shell\\_2\\_2.pdf](https://uefi.org/sites/default/files/resources/UEFI_Shell_2_2.pdf).
- [105] UEFI. 2022. UEFI Platform Initialization Specification. <https://uefi.org/specs/PI/1.8/index.html#>.
- [106] UEFI. 2022. Unified Extensible Firmware Interface (UEFI) Specification Release 2.10. [https://uefi.org/sites/default/files/resources/UEFI\\_Spec\\_2\\_10\\_Aug29.pdf](https://uefi.org/sites/default/files/resources/UEFI_Spec_2_10_Aug29.pdf).
- [107] UEFI. 2023. Unified Extensible Firmware Interface Forum. <https://uefi.org/specifications>.

- [108] UEFI. 2023. Unified Extensible Firmware Interface Forum. <https://uefi.org/about>.
- [109] Dong Wang and Wei Yu Dong. 2019. Attacking Intel UEFI by Using Cache Poisoning. In *Journal of Physics: Conference Series*, Vol. 1187. IOP Publishing, 042072.
- [110] wikileaks. 2017. DerStarke v1.4. [https://wikileaks.org/vault7/darkmatter/document/DerStarke\\_v1\\_4\\_DOC/](https://wikileaks.org/vault7/darkmatter/document/DerStarke_v1_4_DOC/)
- [111] wikileaks. 2017. DerStarke v1.4 RC1 - IVVRR Checklist. [https://wikileaks.org/vault7/darkmatter/document/DerStarke\\_v1\\_4\\_RC1\\_IVVRR\\_Checklist/](https://wikileaks.org/vault7/darkmatter/document/DerStarke_v1_4_RC1_IVVRR_Checklist/)
- [112] Richard Wilkins and Brian Richardson. 2013. UEFI secure boot in modern computer security solutions. In *UEFI forum*. 1–10.
- [113] Cliff Wootton and Cliff Wootton. 2016. Serial Peripheral Interface (SPI). *Samsung ARTIK Reference: The Definitive Developers Guide* (2016), 335–349.
- [114] Zhenkun Yang, Yuriy Viktorov, Jin Yang, Jiewen Yao, and Vincent Zimmer. 2020. Uefi firmware fuzzing with simics virtual platform. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [115] Jiewen Yao, Vincent J Zimmer, and Star Zeng. 2017. A tour beyond BIOS: Using IOMMU for DMA protection in UEFI firmware. *Intel Corporation* 1497 (2017).
- [116] Yilin Zhou, Guojun Peng, Zichuan Li, and Side Liu. 2024. A survey on the evolution of bootkits attack and defense techniques. *China Communications* 21, 1 (2024), 102–130.
- [117] Vincent Zimmer and Michael Krau. 2016. Establishing the Root of Trust. *UEFI.org document dated August* (2016).
- [118] Vincent Zimmer and Mike Rothman. 2011. Intel Technology Journal - UEFI Today: Bootstrapping the Continuum. *Intel Technology Journal* 15 (10 2011).
- [119] Vincent Zimmer, Michael Rothman, and Suresh Marisetty. 2017. *Beyond BIOS: developing with the unified extensible firmware interface*. Walter de Gruyter GmbH & Co KG.